

4장

Linked Lists

4.1	정수 linked list	78
4.2	객체 linked list	79
4.3	Linked list iterator	80
4.4	원형 Linked list	81
4.5	Available list를 사용한 객체 원형 리스트	91
4.6	Doubly linked list	102
4.7	Linked list를 사용한 Polynomial 처리	106
4.8	Available list를 사용한 Polynomial 처리	114
4.9	stack과 queue의 linked list 구현	129
4.10	Generalized list	136
4.11	Heterogeneous list	146



4장

Linked Lists

4.1 정수 linked list

ordered lists를 위한 sequential mapping에 해당하는 array 표현 방법은 임의 elements의 insertion과 deletion 처리 시에 나머지 elements를 이동해야 하는 문제 때문에 성능이 떨어진다. 반면에 linked representation은 pointer 또는 참조(reference) 변수를 사용하여 elements가 더 이상 sequential order로 저장되지 않아도 sorting 된 결과 유지할 수 있게 한다.



그림 4.1 Singly Linked List.

sequential list보다 linked lists를 사용하는 것이 임의 elements의 insert, delete가 더 쉬운 이유는 Node의 link만 변경하면 되기 때문이다. 대신에 array 처럼 기존 elements를 모두 이동해야 하는 불편은 없으나 주어진 값을 찾기 위해서 해당 노드를 순차적으로 따라가는 overhead가 있다. 그림 4.1은 각 노드가 data와 link로 구성되어 있고 데이터 값이 올림차순으로 정렬되는 리스트를 보여 주고 있다. 그림 4.2는 연결 리스트에서 GAT를 삭제시에 FAT를 갖고 있는 노드가 HAT을 갖는 노드로 link를 변경하여 처리한다. Singly linked list는 각 노드의 link가 하나만 포함한 것을 말한다.

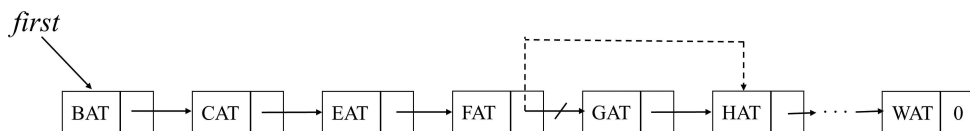


그림 4.2 Singly Linked List에서 삭제 문제.

연결 리스트를 처음 구현하는 코딩 실습으로 노드의 데이터 값이 정수인 경우를 대상으로 구현한다. 노드 값이 올림차순으로 정렬되도록 Add()함수, 주어진 값을 갖는 노드를 찾아 삭제하는 Delete() 함수를 class LinkedList의 멤버함수로 구현한다. 그리고 두 개의 리스트를 merge하는 함수를 operator+() 함수로 구현한다.

Linked list는 데이터를 표현하는 Node를 정의하는 클래스와 Linked List를 정의하는 클래스로 표현한다. class Node의 private data member는 데이터 값을 표현하는 data와 다음 노드를 가리키는 link로 구성된다.

```
class Node {
private:
    int data;
    Node *link;
};
```

class LinkedList는 첫 번째 노드를 가리키는 first를 class Node를 사용하여 data member로 선언한다. class linkedList의 public 함수인 Add(), Delete()에서 노드를 추가 또는 삭제하기 위해서는 class Node에서 friend 선언이 필요하다. first는 LinkedList의 private data member로 선언되었으며 class의 외부에서 접근 또는 변경을 못하게 한다.

```
class LinkedList; // forward 선언
class Node {
    friend class LinkedList;
private:
    int data;
    Node *link;
};
```

```
class LinkedList {
    Node* first;
public:
    LinkedList() {
        first = 0;
    }
    bool Delete(int);
    void Show();
    void Add(int element); //정렬되도록 구현
    bool Search(int data);
```

```

        LinkedList& operator+(LinkedList&);
};

```

```

//소스 코드4.1: 정수 Linked List
/*
1단계-정수 연결 리스트: 단순한 linked list에서 add, delete하는 알고리즘을 코딩
*/
#include <iostream>
#include <time.h>
using namespace std;

class Node {
    friend class LinkedList;
    int data;
    Node* link;
public:
    Node(int element) {
        data = element;
        link = 0;
    }
};

class LinkedList {
    Node* first;
public:
    LinkedList() {
        first = 0;
    }
    bool Delete(int);
    void Show();
    void Add(int element); //정렬되도록 구현
    bool Search(int data);
    LinkedList& operator+(LinkedList&);
};

```

```

};

void LinkedList::Show() { // 전체 리스트를 순서대로 출력한다.
    Node *p = first;
    int num = 0;
    while (p != 0) {
        cout << p->data;
        if (p->link != 0)
            cout << " -> ";
        p = p->link;
    }
    cout << endl;
}

void LinkedList::Add(int element) // 임의 값을 삽입할 때 리스트가 오름차순으로 정렬
이 되도록 한다
{
    Node *newNode = new Node(element);
    if (first == 0) // insert into empty list
    {
        first = newNode;
        return;
    }
    // 기존 노드들이 있으면 순서가 유지하도록 삽입할 노드 위치를 찾는다
    Node *p = first, *q = 0;
    while (p != 0) {
        if (p->data < element) {
            q = p;
            p = p->link;
            if (p == 0) {
                q->link = newNode;
                return;
            }
        }
        else {
            newNode->link = p;
            if (q != 0)
                q->link = newNode;
            else

```

```

        first = newNode;
        return;
    }
}

bool LinkedList::Search(int data) { // 전체 리스트를 순서대로 출력한다.
    Node *ptr = first;
    while (ptr != 0) {
        if (ptr->data == data)
            return true;
        ptr = ptr->link;
    }
    return false;
}

bool LinkedList::Delete(int element) // delete the element
{
    Node *q, *current = first;
    q = current;
    while (current != 0) {
        if (current->data == element) // 삭제
        {
            if (q == first) {
                first = current->link;
                delete current;
                return true;
            }
            q->link = current->link;
            delete current;
            return true;
        }
        else {
            q = current;
            current = current->link;
        }
    }
}

```

```

        return false; // 삭제할 대상이 없다.
    }
    LinkedList& LinkedList::operator+(LinkedList& lb) {
        LinkedList lc;
        Node* a = first, * b = lb.first;

        while (a && b) { //current nodes are not null
            if (a->data == b->data) {
                lc.Add(a->data);
                a = a->link; b = b->link; //advance to next term
            }
            else if (a->data < b->data) {
                lc.Add(a->data);
                a = a->link;
            }
            {
                lc.Add(b->data);
                b = b->link;
            }
        }
        while (a != 0) { //copy rest of a
            lc.Add(a->data);
            a = a->link;
        }
        while (b != 0) { //copy rest of b
            lc.Add(b->data);
            b = b->link;
        }
        return lc;
    }
}
enum Enum {
    Add1, Add2, Delete, Show, Search, Merge, Exit
};
void main() {
    Enum menu; // 메뉴
    int selectMenu;
    int num = 0; bool result = false;

```



```

srand(time(NULL));
LinkedList la, lb, lc;
int data = 0;
do {
    cout << "0.ADD0, 1. Add1, 2.Delete, 3.Show, 4.Search,
5. Merge, 6. Exit 선택::";
    cin >> selectMenu;
    switch (static_cast<Enum>(selectMenu)) {
    case Add1:
        data = rand()% 49;
        la.Add(data);
        break;
    case Add2:
        data = rand() % 49;
        lb.Add(data);
        break;
    case Delete:
        cin >>data;
        result = la.Delete(data);
        if (result)
            cout<<"삭제 완료";
        break;
    case Show:
        cout << "리스트 la = ";
        la.Show();
        cout << "리스트 lb = ";
        lb.Show();
        break;
    case Search: // 회원 번호 검색
        int n; cin >> n;
        result = la.Search(n);
        if (!result)
            cout<<"검색 값 = " <<n <<" 데이터가 없습
니다.";
        else
            cout<<"검색 값 = " << n << " 데이터가 존
재합니다.";
    }
} while (selectMenu != 6);

```

```

        break;
    case Merge:
        lc = la + lb;
        cout << "리스트 lc = ";
        lc.Show();
        break;
    case Exit:
        break;
    }
} while (static_cast<Enum>(selectMenu) != Exit);
cin >> num;
}

```

4.2 객체 linked list

정수 연결 리스트를 객체 버전으로 만드는 소스코드를 구현한다. 소스코드 4.1에서 구현된 node의 데이터가 정수가 아닌 객체로 바꾼다. 2단계 객체 연결 리스트 구현은 소스코드 4.2와 같다. 객체들의 정렬은 `bool operator<(Employee&);`으로 비교한 순위로 정렬한다.

```

//소스코드 4.2 객체 linked list
/*
2단계- 객체 연결 리스트: 단순한 linked list에서 insert는 올림차순으로 정렬되도록 처리, delete하는 알고리즘을 코딩
*/
#include <iostream>
#include <time.h>
using namespace std;
class Employee {
    friend class Node;
    friend class LinkedList;
    string eno;
    string ename;

```

```

public:
    Employee() {}
    Employee(string sno, string sname) :eno(sno), ename(sname) {}
    friend ostream& operator<<(ostream& os, Employee&);
    bool operator<(Employee&);
    bool operator==(Employee&);
};

ostream& operator<<(ostream& os, Employee& emp) {
    os << " " << emp.eno << emp.ename;
    return os;
}

bool Employee::operator==(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename == emp.ename);
    else
        return false;
}

bool Employee::operator<(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename < emp.ename);
    else
        return (eno < emp.eno);
}

class Node {
    friend class LinkedList;
    Employee data;
    Node* link;
public:
    Node(Employee element) {
        data = element;
        link = nullptr;
    }
};

class LinkedList {

```

```

        Node* first;
public:
    LinkedList() {
        first = nullptr;
    }
    bool Delete(string);
    void Show();
    void Add(Employee); //sno로 정렬되도록 구현
    bool Search(string);
    LinkedList& operator+(LinkedList&);
};

void LinkedList::Show() { // 전체 리스트를 순서대로 출력한다.
    Node* p = first;
    while (p != nullptr) {
        cout << p->data;
        if (p->link != nullptr)
            cout<<" -> ";
        p = p->link;
    }
    cout << endl;
}

void LinkedList::Add(Employee element) // 임의 값을 삽입할 때 리스트가 오름차순으
로 정렬이 되도록 한다
{
    Node* newNode = new Node(element);
    if (first == nullptr) // insert into empty list
    {
        first = newNode;
        return;
    }
    // 기존 노드들이 있으면 순서가 유지하도록 삽입할 노드 위치를 찾는다
    Node* p = first, * q = nullptr;
    while (p != nullptr) {
        if (p->data < element) { //operator<() 구현 필요
            q = p;
            p = p->link;
        }
        if (p == nullptr) {

```

```

        q->link = newNode;
        return;
    }
}
else {
    newNode->link = p;
    if (q != nullptr)
        q->link = newNode;
    else
        first = newNode;
    return;
}
}

bool LinkedList::Search(string eno) { // sno를 갖는 레코드를 찾기
    Node* ptr = first;
    while (ptr != nullptr) {
        if (ptr->data.eno == eno)
            return true;
        ptr = ptr->link;
    }
    return false;
}

bool LinkedList::Delete(string eno) // delete the element
{
    Node* q, * current = first;
    q = current;
    while (current != nullptr) {
        if (current->data.eno == eno) // 삭제
        {
            if (q == first) {
                first = current->link;
                delete current;
                return true;
            }
            q->link = current->link;
            delete current;

```

```

        return true;

    }
    else {
        q = current;
        current = current->link;
    }

}

return false; // 삭제할 대상이 없다.
}

LinkedList& LinkedList::operator+(LinkedList& lb) {
    LinkedList lc;
    Node* a = first, * b = lb.first;

    while (a && b) { //current nodes are not null
        if (a->data == b->data) {
            lc.Add(a->data);
            a = a->link; b = b->link; //advance to next term
        }
        else if (a->data < b->data) {
            lc.Add(a->data);
            a = a->link;
        }
        {
            lc.Add(b->data);
            b = b->link;
        }
    }

    while (a != 0) { //copy rest of a
        lc.Add(a->data);
        a = a->link;
    }

    while (b != 0) { //copy rest of b
        lc.Add(b->data);
        b = b->link;
    }
}

```

```

        return lc;
    }
    enum Enum {
        Add1, Add2, Delete, Show, Search, Merge, Exit
    };

    void main() {
        Enum menu; // 메뉴
        int selectMenu, num;
        string eno,ename;
        bool result = false;
        LinkedList la, lb, lc;
        Employee *data;
        do {
            cout << "0.ADD0, 1. Add1, 2.Delete, 3.Show, 4.Search, 5. Merge,
6. Exit 선택::";

            cin >> selectMenu;
            switch (static_cast<Enum>(selectMenu)) {
            case Add1:
                cout << "사원번호 입력:: ";
                cin >> eno;
                cout << "사원 이름 입력:: ";
                cin >> ename;
                data = new Employee(eno, ename);
                la.Add(*data);
                break;
            case Add2:
                cout << "사원번호 입력:: ";
                cin >> eno;
                cout << "사원 이름 입력:: ";
                cin >> ename;
                data = new Employee(eno, ename);
                lb.Add(*data);
                break;
            case Delete:
                cout << "사원번호 입력:: ";
                cin >> eno;

```

```

        result = la.Delete(eno);
        if (result)
            cout << "삭제 완료";
        break;
    case Show:
        cout << "리스트 la = ";
        la.Show();
        cout << "리스트 lb = ";
        lb.Show();
        break;
    case Search:
        cout << "사원번호 입력:: ";
        cin >> eno;
        result = la.Search(eno);
        if (!result)
            cout << "검색 값 = " << eno << " 데이터가 없습니
다.";
        else
            cout << "검색 값 = " << eno << " 데이터가 존재합
니다.";
        break;
    case Merge:
        lc = la + lb;
        cout << "리스트 lc = ";
        lc.Show();
        break;

    case Exit: // 꼬리 노드 삭제
        break;
    }
} while (static_cast<Enum>(selectMenu) != Exit);
cin >> num;
}

```

4.3 linked list iterator

container class 에 대한 iterators 의 사용 예로서 1) class C 의 모든 integers를 print, 2) C에 있는 모든 정수의 max, min, mean, median 을 구한다, 3) C에 있는 모든 정수의 sum, product, sum of squares를 구한다, 4) 어떤 조건 P 를 만족하는 모든 정수를 구한다, 5) 주어진 f(x)가 max가 되게 하는 정수 x를 구하는 함수를 작성할 때 iterator 객체를 사용한다.

iterator 객체는 container class의 모든 elements를 one by one으로 traverse 하는 pointer를 내부 변수로 포함한다. container class인 LinkedList의 모든 elements를 traverse하는 class ListIterator는 private data member로서 list와 current를 포함한다[1]. 데이터 멤버 list는 iterator의 적용 대상인 container class 객체를 상수로서 참조변수로 갖고 있게 된다. 데이터 멤버 current는 pointer 변수로서 Next()함수가 호출될 때마다 다음 노드를 가리키게 된다. Iterator 객체를 사용하여 리스트의 merge, min, max, sum 등의 함수를 소스코드 4.3이 보여주고 있다.

두 개의 linked list를 merge하는 `LinkedList& LinkedList::operator+(LinkedList& lb)`을 iterator를 사용한 구현과 iterator 없이 pointer를 사용한 구현 실습을 수행한다. `operator+()` 함수 구현시에 iterator를 사용한 구현이 좀 더 간편한 구현임을 느끼는 것이 중요하다. `LinkedList::operator+()` 구현시에 `LinkedList::Add()` 대신에 `LinkedList::Attach()`를 사용한 구현 실습을 수행한다. `Attach()` 함수는 현재 리스트의 마지막에 추가하는 함수로서 append 기능을 수행하도록 구현한다.

소스코드 4.3 - 객체 연결 리스트의 iterator

```
/*
3단계-객체 연결 리스트의 iterator 버전
*/
#include <iostream>
#include <time.h>
using namespace std;
class Employee {
    friend class Node;
    friend class LinkedList;
    friend class ListIterator;
private:
    string eno;
```

```

        string ename;
        int salary;
public:
    Employee() {}
    Employee(string sno, string sname, int salary) :eno(sno), ename(sname),
salary(salary) {}
        friend ostream& operator<<(ostream& os, Employee&);
        bool operator<(Employee&);
        bool operator==(Employee&);
        char compare(const Employee*)const;
        int getSalary() const {
            return salary;
        }
};
ostream& operator<<(ostream& os, Employee& emp) {
    os << "<" << emp.eno<< ", " << emp.ename<< ", " << emp.salary<< ">";
    return os;
}
bool Employee::operator==(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename == emp.ename);
    else
        return false;
}
bool Employee::operator<(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename < emp.ename);
    else
        return (eno < emp.eno);
}
char Employee::compare(const Employee* emp) const {
    if (this->eno > emp->eno)
        return '>';
    else if (this->eno < emp->eno)
        return '<';
}

```

```

        else
            return '=';
    }
class Node {
    friend class ListIterator;
    friend class LinkedList;
    Employee data;
    Node* link;
public:
    Node() {}
    Node(Employee element) {
        data = element;
        link = nullptr;
    }
};
class ListIterator;
class LinkedList {
    friend class ListIterator;
    Node* first;
public:
    LinkedList() {
        first = nullptr;
    }
    bool Delete(string);
    void Show();
    void Add(Employee); //sno로 정렬되도록 구현
    bool Search(string);
    LinkedList& operator+(LinkedList&);
};
class ListIterator {
public:
    ListIterator(const LinkedList& lst);
    ~ListIterator();
    bool NotNull();
    bool NextNotNull();
    Employee* First();

```

```

Employee* Next();
Employee& operator*() const;
Employee* operator->()const;
ListIterator& operator++(); //Next()
ListIterator operator ++(int);
bool operator != (const ListIterator) const;
bool operator == (const ListIterator) const;
Employee* GetCurrent();
private:
    Node* current; //pointer to nodes
    const LinkedList& list;//existing list
};
ListIterator::ListIterator(const LinkedList& lst) : list(lst), current(lst.first) {
    cout << "List Iterator is constructed" << endl;
}
void LinkedList::Show() { // 전체 리스트를 순서대로 출력한다.
    Node* p = first;
    while (p != nullptr) {
        cout << p->data;
        if (p->link != nullptr)
            cout << " -> ";
        p = p->link;
    }
    cout << endl;
}
void LinkedList::Add(Employee element) // 임의 값을 삽입할 때 리스트가 오름차순으
로 정렬이 되도록 한다
{
    Node* newNode = new Node(element);
    if (first == nullptr) // insert into empty list
    {
        first = newNode;
        return;
    }
    // 기존 노드들이 있으면 순서가 유지하도록 삽입할 노드 위치를 찾는다
    Node* p = first, * q = nullptr;
    while (p != nullptr) {

```

```

        if (p->data < element) { //operator<() 구현 필요
            q = p;
            p = p->link;
            if (p == nullptr) {
                q->link = newNode;
                return;
            }
        }
        else {
            newNode->link = p;
            if (q != nullptr)
                q->link = newNode;
            else
                first = newNode;
            return;
        }
    }
}

bool LinkedList::Search(string eno) { // sno를 갖는 레코드를 찾기
    Node* ptr = first;
    while (ptr != nullptr) {
        if (ptr->data.eno == eno)
            return true;
        ptr = ptr->link;
    }
    return false;
}

bool LinkedList::Delete(string eno) // delete the element
{
    Node* q, * current = first;
    q = current;
    while (current != nullptr) {
        if (current->data.eno == eno) // 삭제
        {
            if (q == first) {
                first = current->link;
                delete current;
            }
        }
    }
}

```

```

        return true;
    }
    q->link = current->link;
    delete current;
    return true;
}
else {
    q = current;
    current = current->link;
}

}
return false; // 삭제할 대상이 없다.
}
LinkedList& LinkedList::operator+(LinkedList& lb) {
    Employee* p, * q;
    ListIterator Aiter(*this); ListIterator Biter(lb);
    LinkedList lc;
    p = Aiter.First(); q = Biter.First();
    while (Aiter.NotNull() && Biter.NotNull()) {
        switch (p->compare(q)) {
            case '=':
                lc.Add(*p);
                p = Aiter.Next(); q = Biter.Next();
                break;
            case '<':
                lc.Add(*p);
                p = Aiter.Next();
                break;
            case '>':
                lc.Add(*q);
                q = Biter.Next();
                break;
        }
    }
}

```

```

        while (Aiter.NotNull()) {
            lc.Add(*p);
            p = Aiter.Next();
        }
        while (Biter.NotNull()) {
            lc.Add(*q);
            q = Biter.Next();
        }
        return lc;
    }

    bool ListIterator::NotNull() {
        if (current != NULL)
            return true;
        else
            return false;
    }

    bool ListIterator::NextNotNull() {
        if (current->link != NULL)
            return true;
        else
            return false;
    }

    Employee* ListIterator::First() {
        return &list.first->data;
    }

    Employee* ListIterator::Next() {
        current = current->link;
        Employee* e = &current->data;
        return e;
    }

    Employee* ListIterator::GetCurrent() {
        return &current->data;
    }

    ListIterator::~ListIterator() {

```

```

}

Employee& ListIterator::operator*() const {
    return current->data;
}

Employee* ListIterator::operator->()const {
    return &current->data;
}

ListIterator& ListIterator::operator++() {
    current = current->link;
    return *this;
}

ListIterator ListIterator::operator ++(int) {
    ListIterator old = *this;
    current = current->link;
    return old;
}

bool ListIterator::operator != (const ListIterator right) const {
    return current != right.current;
}

bool ListIterator::operator == (const ListIterator right) const {
    return current == right.current;
}

//int printAll(const List& l);//list iterator를 사용하여 작성하는 연습
//int sumProductFifthElement(const List& l);//list iterator를 사용하여 작성하는 연
습
int sum(const LinkedList& l)//올바른지 코드 점검이 필요함
{
    ListIterator li(l);

    if (!li.NotNull()) return 0;
    Employee* emp = li.First();

```



```

int retValue = emp->getSalary();
while (li.NextNotNull() == true) {
    ++li; //current를 증가시킴
    //retvalue = retvalue + *li.Next();
    retValue = retValue + li.GetCurrent()->getSalary(); //현재 current
    가 가르키는 node의 값을 가져옴
}
return retValue;
}

```

```

double avg(const LinkedList& l) //올바른지 코드 점검이 필요함
{
    ListIterator li(l);
    if (!li.NotNull()) return 0;
    int retvalue = li.First()->getSalary();
    int count = 1;
    while (li.NextNotNull() == true) {
        ++li;
        count++;
        //retvalue = retvalue + *li.Next();
        retvalue = retvalue + li.GetCurrent()->getSalary();
    }

    double result = (double)retvalue / count;

    return result;
}

```

```

int min(const LinkedList& l) //올바른지 코드 점검이 필요함
{
    ListIterator li(l);
    if (!li.NotNull()) return -1;
    int minValue = li.First()->getSalary();

    while (li.NextNotNull() == true) {
        ++li;
        if (minValue > li.GetCurrent()->getSalary())

```

```

        {
            minValue = li.GetCurrent()->getSalary();
        }
    }
    return minValue;
}

```

```

int max(const LinkedList& l)//올바른지 코드 점검이 필요함
{

```

```

    ListIterator li(l);
    if (!li.NotNull()) return -1;
    int maxValue = li.First()->getSalary();

    while (li.NextNotNull() == true) {
        ++li;
        if (maxValue < li.GetCurrent()->getSalary())
        {
            maxValue = li.GetCurrent()->getSalary();
        }
    }
    return maxValue;
}

```

```

enum Enum {
    Add0, Add1, Delete, Show, Search, Merge, SUM, AVG, MIN, MAX, Exit
};

```

```

void main() {
    Enum menu; // 메뉴
    int selectMenu, num;
    string eno, ename;
    int pay;
    Employee* data;
    bool result = false;
    LinkedList la, lb, lc;
    do {
        cout <<endl<< "0.Add0, 1.Add1, 2.Delete, 3.Show, 4.Search,

```

5.Merge, 6. sum, 7.avg, 8.min, 9.max, 10.exit 선택::";

```
cin >> selectMenu;
switch (static_cast<Enum>(selectMenu)) {
case Add0:
    cout << "사원번호 입력:: ";
    cin >> eno;
    cout << "사원 이름 입력:: ";
    cin >> ename;
    cout << "사원 급여:: ";
    cin >> pay;
    data = new Employee(eno, ename, pay);
    la.Add(*data);
    break;
case Add1:
    cout << "사원번호 입력:: ";
    cin >> eno;
    cout << "사원 이름 입력:: ";
    cin >> ename;
    cout << "사원 급여:: ";
    cin >> pay;
    data = new Employee(eno, ename, pay);
    lb.Add(*data);
    break;
case Delete:
    cout << "사원번호 입력:: ";
    cin >> eno;
    result = la.Delete(eno);
    if (result)
        cout << "삭제 완료";
    break;
case Show:
    cout << "리스트 la = ";
    la.Show();
    cout << "리스트 lb = ";
    lb.Show();
    break;
case Search:
```

```

        cout << "사원번호 입력:: ";
        cin >> eno;
        result = la.Search(eno);
        if (!result)
            cout << "검색 값 = " << eno << " 데이터가 없습니
다.";

        else
            cout << "검색 값 = " << eno << " 데이터가 존재합
니다.";

        break;
    case Merge:
        lc = la + lb;
        cout << "리스트 lc = ";
        lc.Show();
        break;
    case SUM:  cout << "sum() = " << sum(la) << endl; break;
    case AVG:  cout << "avg() = " << avg(la) << endl; break;
    case MIN:  cout << "min() = " << min(la) << endl; break;
    case MAX:  cout << "max() = " << max(la) << endl; break;
    case Exit: // 꼬리 노드 삭제
        break;
    }
} while (static_cast<Enum>(selectMenu) != Exit);
cin >> num;
}

```

4.4 원형 linked list

circular list는 마지막 노드가 첫 번째 노드를 가리키는 linked list이다. circular list를 사용하는 이유는 list 전체를 available list에 반환할 때 모든 노드를 방문하지 않아도 전체 list를 반환할 수 있는 이점이 있기 때문이다. circular list를 사용하여 두개의 list를 merge 할 때 head node를 포함하고 있으면 list가 null인지 확인하는 if 문장을 줄일 수 있어 알고리즘이 더 간단해진다. circular list를 사용할 때 리스트 객체의 데이터 멤버가 그림 4.3처럼 첫 번째 노드를 가리키는 first를 사용하거나 그림 4.4처럼 마지막 노드를 가리키는 last를 데이터 멤버가 가

질 수 있다.

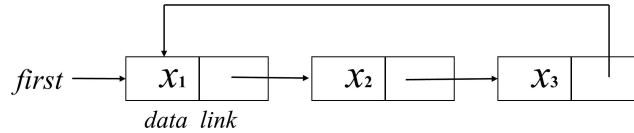


그림 4.3 Circular List.

circular list의 private data member인 first가 첫 번째 노드를 가리키는 경우에는 x_1 보다 작은 값 또는 x_3 보다 큰 값을 insert하려면 원형을 만들어야 하므로 list를 traverse하여 마지막 노드를 찾아야 한다. 반면에 private data member를 last로 하고 마지막 노드를 가리키면 x_1 보다 작은 값 또는 x_3 보다 큰 값을 insert하는 알고리즘이 간단해진다[1].

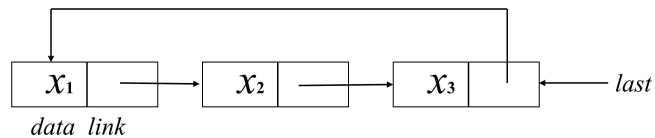


그림 4.4 last node를 접근하는 Circular List.

empty list도 head node를 사용하여 그림 4.5와 같은 빈 circular list를 만들면 merge하는 알고리즘을 간단하게 구현할 수 있다. circular list가 dummy head 노드를 포함하면 list가 null인지 확인하는 if 문을 사용할 필요가 없게 된다.

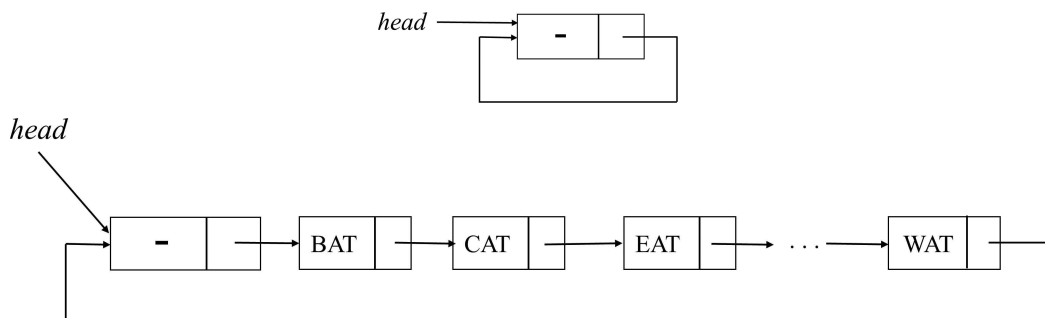


그림 4.5 head node가 있는 circular list.

circular list에 대한 iterator도 구현하여 사용할 수가 있다. 그리고 이러한 원형 리스트는 doubly linked list도 circular list이면서 head node를 갖도록 확장 구현할 수 있다. 원형 리스트에 대한 operator+()의 구현시세 Add() 함수 대신에

append 기능을 하는 Attach() 함수를 만들어 마지막 순위에 추가하도록 구현한다.

```
//소스 코드4.4: Circular Linked List
```

```
/*
4단계- 원형 객체 연결 리스트의 iterator 버전
CircularList를 대상으로 한 iterator를 구현한다.
*/
#include <iostream>
#include <time.h>
using namespace std;

class Employee {
    friend class Node;
    friend class CircularList;
    friend class ListIterator;
private:
    string eno;
    string ename;
    int salary;
public:
    Employee() {}
    Employee(string sno, string sname, int salary) :eno(sno), ename(sname),
salary(salary) {}
    friend ostream& operator<<(ostream& os, Employee&);
    bool operator<(Employee&);
    bool operator==(Employee&);
    char compare(const Employee* emp) const;
    int getSalary() const {
        return salary;
    }
};

ostream& operator<<(ostream& os, Employee& emp) {
    os << "<" << emp.eno << ", " << emp.ename << ", " << emp.salary << ">";
    return os;
}
```

```

}
bool Employee::operator==(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename == emp.eno);
    else
        return false;
}
bool Employee::operator<(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename < emp.eno);
    else
        return (eno < emp.eno);
}
char Employee::compare(const Employee* emp) const {
    if (this->eno > emp->eno)
        return '>';
    else if (this->eno < emp->eno)
        return '<';
    else
        return '=';
}
}
class Node {
    friend class ListIterator;
    friend class CircularList;
    Employee data;
    Node* link;
public:
    Node(){}
    Node(Employee element) {
        data = element;
        link = nullptr;
    }
};

```

```

class CircularList {
    friend class ListIterator;
    Node* first; //last 노드를 가리키는 last 변수를 사용하는 버전으로 구현 실습
    //static Node* av;
public:
    CircularList() {
        first = new Node(); first->link = first;
    }
    bool Delete(string);
    void Show();
    void Add(Employee*); //sno로 정렬되도록 구현
    bool Search(string);
    CircularList& operator+(CircularList&);
    friend ostream& operator<<(ostream& os, CircularList& l);
};

class ListIterator {
    //원형 리스트의 head node와 원형이므로 리스트 마지막 노드에 도달시 수정이
    필요하다
public:
    ListIterator(const CircularList& lst);
    ~ListIterator();
    bool NotNull();
    bool NextNotNull();
    Employee* First();
    Employee* Next();
    Employee& operator*() const;
    Employee* operator->()const;
    ListIterator& operator++(); //Next()
    ListIterator operator ++(int);
    bool operator != (const ListIterator) const;
    bool operator == (const ListIterator) const;
    Employee* GetCurrent();
private:
    Node* current; //pointer to nodes
    const CircularList& list; //existing list
};

```



```

ostream& operator<<(ostream& os, CircularList& l)
{
    ListIterator li(l);
    if (!li.NotNull()) return os;
    os << *li.First();
    while (li.NextNotNull())
        os << " + " << *li.Next();
    os << endl;
    return os;
}

void CircularList::Show() { // 전체 리스트를 순서대로 출력한다.
    Node* p = first->link;
    while (p != first) {
        cout << p->data;
        if (p->link != first)
            cout << " -> ";
        p = p->link;
    }
    cout << endl;
}

void CircularList::Add(Employee* element) // 임의 값을 삽입할 때 리스트가 오름차
순으로 정렬이 되도록 한다
{
    Node* newNode = new Node(*element);
    Node* p = first->link, * q = first;
    if (p == first) // insert into empty list
    {
        p->link = newNode; newNode->link = first;
        return;
    }
    // 기존 노드들이 있으면 순서가 유지하도록 삽입할 노드 위치를 찾는다

    while (p != first) {
        if (p->data < *element) { //operator<() 구현 필요
            q = p;
            p = p->link;
            if (p == first) {

```

```

        q->link = newNode; newNode->link = first;
        return;
    }
}
else {
    newNode->link = p;
    q->link = newNode;
    return;
}
}
}

bool CircularList::Search(string eno) { // sno를 갖는 레코드를 찾기
    Node* p = first->link;
    while (p != first) {
        if (p->data.eno == eno)
            return true;
        p = p->link;
    }
    return false;
}

bool CircularList::Delete(string eno) // delete the element
{
    Node* p = first->link;
    Node* q = first;
    while (p != first) {
        if (p->data.eno == eno) // 삭제
        {
            if (p->link == first) {
                q->link = first;
                delete p;
                return true;
            }
            q->link = p->link;
            delete p;
            return true;
        }
    }
}

```

```

        else {
            q = p;
            p = p->link;
        }
    }
    return false; // 삭제할 대상이 없다.
}

CircularList& CircularList::operator+(CircularList& lb) {
    Employee* p, * q;
    ListIterator Aiter(*this); ListIterator Biter(lb);
    CircularList lc;
    p = Aiter.First(); q = Biter.First();
    while (Aiter.NotNull() && Biter.NotNull()) {
        switch (p->compare(q)) {
            case '=':
                lc.Add(p);
                p = Aiter.Next(); q = Biter.Next();
                break;
            case '<':
                lc.Add(p);
                p = Aiter.Next();
                break;
            case '>':
                lc.Add(q);
                q = Biter.Next();
                break;
        }
    }
    while (Aiter.NotNull()) {
        lc.Add(p);
        p = Aiter.Next();
    }
    while (Biter.NotNull()) {
        lc.Add(q);
        q = Biter.Next();
    }
}

```

```

    }
    return lc;
}

ListIterator::ListIterator(const CircularList& lst) : list(lst), current(lst.first->link) {
    cout << "List Iterator is constructed" << endl;
}

bool ListIterator::NotNull() {
    if (current != list.first)
        return true;
    else
        return false;
}

bool ListIterator::NextNotNull() {
    if (current->link != list.first)
        return true;
    else
        return false;
}

Employee* ListIterator::First() {
    return &list.first->link->data;
}

Employee* ListIterator::Next() {
    current = current->link;
    Employee* e = &current->data;
    return e;
}

Employee* ListIterator::GetCurrent() {
    return &current->data;
}

ListIterator::~ListIterator() {
}

Employee& ListIterator::operator*() const {
    return current->data;
}

```

```

}

Employee* ListIterator::operator->()const {
    return &current->data;
}

ListIterator& ListIterator::operator++() {
    current = current->link;
    return *this;
}

ListIterator ListIterator::operator ++(int) {
    ListIterator old = *this;
    current = current->link;
    return old;
}

bool ListIterator::operator != (const ListIterator right) const {
    return current != right.current;
}

bool ListIterator::operator == (const ListIterator right) const {
    return current == right.current;
}

//int printAll(const List& l);//list iterator를 사용하여 작성하는 연습
//int sumProductFifthElement(const List& l);//list iterator를 사용하여 작성하는 연
습
int sum(const CircularList& l)
{
    ListIterator li(l);

    if (!li.NotNull()) return 0;
    Employee* emp = li.First();
    int retValue = emp->getSalary();
    while (li.NextNotNull() == true) {
        ++li;//current를 증가시킴
        //retvalue = retvalue + *li.Next();
    }
}

```

```

        retValue = retValue + li.GetCurrent()->getSalary();//현재 current
가 가르키는 node의 값을 가져옴
    }
    return retValue;
}

```

```

double avg(const CircularList& l)
{
    ListIterator li(l);
    if (!li.NotNull()) return 0;
    int retvalue = li.First()->getSalary();
    int count = 1;
    while (li.NextNotNull() == true) {
        ++li;
        count++;
        //retvalue = retvalue + *li.Next();
        retvalue = retvalue + li.GetCurrent()->getSalary();
    }

    double result = (double)retvalue / count;

    return result;
}

```

```

int min(const CircularList& l)
{
    ListIterator li(l);
    if (!li.NotNull()) return -1;
    int minValue = li.First()->getSalary();

    while (li.NextNotNull() == true) {
        ++li;
        if (minValue > li.GetCurrent()->getSalary())
        {
            minValue = li.GetCurrent()->getSalary();
        }
    }
}

```

```

        return minValue;
    }

int max(const CircularList& l)
{
    ListIterator li(l);
    if (!li.NotNull()) return -1;
    int maxValue = li.First()->getSalary();

    while (li.NextNotNull() == true) {
        ++li;
        if (maxValue < li.GetCurrent()->getSalary())
        {
            maxValue = li.GetCurrent()->getSalary();
        }
    }
    return maxValue;
}

enum Enum {
    Add0, Add1, Delete, Show, Search, Merge, SUM, AVG, MIN, MAX, Exit
};

//Node* CircularList::av = NULL; //static 변수의 초기화 방법을 기억해야 한다

void main() {
    Enum menu; // 메뉴
    int selectMenu, num;
    string eno, ename;
    int pay;
    Employee* data;
    bool result = false;
    CircularList la, lb, lc;
    do {
        cout << endl << "0.Add0, 1.Add1, 2.Delete, 3.Show, 4.Search,
5.Merge, 6. sum, 7.avg, 8.min, 9.max, 10.exit 선택::";
        cin >> selectMenu;
    } while (selectMenu < 10);
}

```

```

switch (static_cast<Enum>(selectMenu)) {
case Add0:
    cout << "사원번호 입력:: ";
    cin >> eno;
    cout << "사원 이름 입력:: ";
    cin >> ename;
    cout << "사원 급여:: ";
    cin >> pay;
    data = new Employee(eno, ename, pay);
    la.Add(data);
    break;
case Add1:
    cout << "사원번호 입력:: ";
    cin >> eno;
    cout << "사원 이름 입력:: ";
    cin >> ename;
    cout << "사원 급여:: ";
    cin >> pay;
    data = new Employee(eno, ename, pay);
    lb.Add(data);
    break;
case Delete:
    cout << "사원번호 입력:: ";
    cin >> eno;
    result = la.Delete(eno);
    if (result)
        cout << "삭제 완료";
    break;
case Show:
    cout << "리스트 la = ";
    la.Show();
    cout << "리스트 lb = ";
    lb.Show();
    break;
case Search:
    cout << "사원번호 입력:: ";
    cin >> eno;

```



```

        result = la.Search(eno);
        if (!result)
            cout << "검색 값 = " << eno << " 데이터가 없습니
다.";

        else
            cout << "검색 값 = " << eno << " 데이터가 존재합
니다.";

        break;
    case Merge:
        lc = la + lb;
        cout << "리스트 lc = ";
        lc.Show();
        break;

    case SUM:  cout << "sum() = " << sum(la) << endl; break;
    case AVG:  cout << "avg() = " << avg(la) << endl; break;
    case MIN:  cout << "min() = " << min(la) << endl; break;
    case MAX:  cout << "max() = " << max(la) << endl; break;
    case Exit: // 꼬리 노드 삭제
        break;
    }
} while (static_cast<Enum>(selectMenu) != Exit);
cin >> num;
}

```

4.5 Available list를 사용한 객체 원형 리스트

$d(x) = a(x) * b(x) + c(x)$ 를 계산하기 위해 $t = a * b$ 를 먼저 구한 후에 $d = t + c$ 를 처리한다. 이 경우에 임시 변수 t 는 linked list로 갖고 있는 데 모든 노드를 반환하는 것이 필요하다.

```

void func( )
{
    LinkedList a, b, c, d, t;

```

```

    cin >> a; // read and create polynomials
    cin >> b;
    cin >> c;
    t = a * b
    d = t + c;
    cout << d;
}

```

다항식 연산을 처리하는 *func* 이 종료되면 모든 list의 객체들은 반환되어야 한다. 즉, 다항식 객체 a, b, c, t, d에 사용된 모든 노드가 반환되어야 한다. 그런데 함수 *func*이 종료되면 destructor가 호출되므로 polynomial 객체들은 없어지나 data member가 가리키는 노드들은 삭제되지 않는다. 즉, new에 의해 생성된 노드들은 delete에 의해 삭제되어야 한다. 따라서 LinkedList 객체들만 삭제하면 노드들은 더 이상 접근이 가능하지 않은 dangling reference가 발생하게 된다.

class List의 destructor는 dangling reference가 발생하지 않도록 하기 위해 모든 노드를 방문하여 delete하는 loop가 필요하게 된다.

```

template <class T>
List<T>::~~List( ) ( ) {
    ListNode<T> *next;
    for (; first; first = next) {
        next = first->link;
        delete first;
    }
}

```

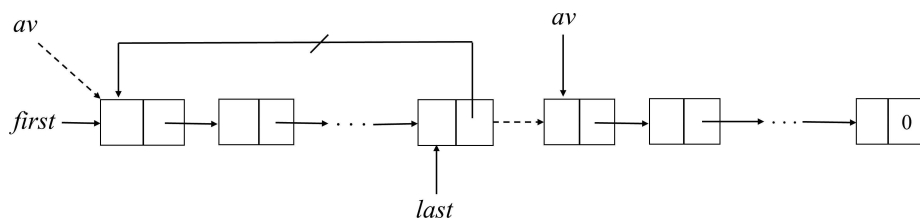


그림 4.6 Available List.

circular list를 이용하여 list의 모든 노드를 반환하는 것을 간단하게 구현할 수 있다. circular lists를 위한 erase algorithm을 간단하게 구현하기 위해서 삭제된

ChainNode들을 *av* list, 즉 available-space list로 유지한다[1]. *av* list가 empty 이면 새로운 ChainNode를 만들기 위해 *new* 을 사용한다. *av*는 `CircList<Type>` 의 static class member로 선언한다.

```
//소스 코드4.5: Available Linked List
// circular list를 사용한 version
/*
5단계- 원형 객체 연결 리스트의 available list, getNode, retNode
head node를 갖고 있고 삭제된 노드들은 available list에 리턴한다.
CircularList를 대상으로 한 iterator를 구현한다.
*/
#include <iostream>
#include <time.h>
using namespace std;

class Employee {
    friend class Node;
    friend class CircularList;
    friend class ListIterator;
private:
    string eno;
    string ename;
    int salary;
public:
    Employee() {}
    Employee(string sno, string sname, int salary) :eno(sno), ename(sname),
salary(salary) {}
    friend ostream& operator<<(ostream& os, Employee&);
    bool operator<(Employee&);
    bool operator==(Employee&);
    char compare(const Employee* emp) const;
    int getSalary() const {
        return salary;
    }
};

ostream& operator<<(ostream& os, Employee& emp) {
    os << "<" << emp.eno << ", " << emp.ename << ", " << emp.salary << ">";
```

```

        return os;
    }
    bool Employee::operator==(Employee& emp) {
        bool result = false;
        if (eno == emp.eno)
            return (ename == emp.eno);
        else
            return false;
    }
    bool Employee::operator<(Employee& emp) {
        bool result = false;
        if (eno == emp.eno)
            return (ename < emp.eno);
        else
            return (eno < emp.eno);
    }
    char Employee::compare(const Employee* emp) const {
        if (this->eno > emp->eno)
            return '>';
        else if (this->eno < emp->eno)
            return '<';
        else
            return '=';
    }
}

class Node {
    friend class ListIterator;
    friend class CircularList;
    Employee data;
    Node* link;
public:
    Node() { }
    Node(Employee element) {
        data = element;
        link = nullptr;
    }
};

```

```

class CircularList {
    friend class ListIterator;
    Node* last; //last 노드를 가리키는 last 변수를 사용하는 버전으로 구현 실습
    static Node* av;
public:
    CircularList() {
        last = new Node(); last->link = last;
    }
    bool Delete(string);
    void Show();
    void Add(Employee*); //sno로 정렬되도록 구현
    bool Search(string);
    Node* GetNode();
    void RetNode(Node*);
    void Erase();
    CircularList& operator+(CircularList&);
    friend ostream& operator<<(ostream& os, CircularList& l);
};

class ListIterator {
public:
    ListIterator(const CircularList& lst);
    ~ListIterator();
    bool NotNull();
    bool NextNotNull();
    Employee* First();
    Employee* Next();
    Employee& operator*() const;
    Employee* operator->() const;
    ListIterator& operator++(); //Next()
    ListIterator operator ++(int);
    bool operator != (const ListIterator) const;
    bool operator == (const ListIterator) const;
    Employee* GetCurrent();
private:
    Node* current; //pointer to nodes
    const CircularList& list; //existing list

```

```
};
```

```
Node* CircularList::GetNode()
{ //provide a node for use
    Node* x;
    if (av) { x = av, av = av->link; }
    else x = new Node;
    return x;
}
```

```
void CircularList::RetNode(Node* x)
{ //free the node pointed to by x
    x->link = av;
    av = x;
    x = 0;
}
```

```
ostream& operator<<(ostream& os, CircularList& l)
{
```

```
    ListIterator li(l);
    if (!li.NotNull()) return os;
    os << *li.First();
    while (li.NextNotNull())
        os << " + " << *li.Next();
    os << endl;
    return os;
}
```

```
void CircularList::Show() { // 전체 리스트를 순서대로 출력한다.
```

```
    if (last == NULL)
        return;
    Node* first = last->link;
    Node* p = first->link;
    while (p != first) {
        cout << p->data;
        if (p != last)
            cout << " -> ";
        p = p->link;
    }
```

```

    }
    cout << endl;
}

void CircularList::Add(Employee* element) // 임의 값을 삽입할 때 리스트가 오름차
순으로 정렬이 되도록 한다
{
    Node* newNode = GetNode(); newNode->data = *element;
    Node* first = last->link;
    Node* p = first->link;
    Node* q = first;
    if (p == first) // insert into empty list
    {
        p->link = newNode; newNode->link = last;
        last = newNode;
        return;
    }
    // 기존 노드들이 있으면 순서가 유지하도록 삽입할 노드 위치를 찾는다

    while (p != first) {
        if (p->data < *element) { //operator<() 구현 필요
            q = p;
            p = p->link;
            if (p == first) {
                q->link = newNode; newNode->link = first; last
= newNode;
                return;
            }
        }
        else {
            newNode->link = p;
            q->link = newNode;
            return;
        }
    }
}

bool CircularList::Search(string eno) { // sno를 갖는 레코드를 찾기
    Node* first = last->link;

```

```

Node* p = first->link;
while (p != first) {
    if (p->data.eno == eno)
        return true;
    p = p->link;
}
return false;
}
bool CircularList::Delete(string eno) // delete the element
{
    Node* first = last->link;
    Node* p = first->link;
    Node* q = first;
    while (p != first) {
        if (p->data.eno == eno) // 삭제
        {
            if (p == last) {
                q->link = first;
                RetNode(p);
                last = q;
                return true;
            }
            q->link = p->link;
            RetNode(p);
            return true;
        }
        else {
            q = p;
            p = p->link;
        }
    }
    return false; // 삭제할 대상이 없다.
}
void CircularList::Erase() {
    Node* temp = last->link;

```



```

        last->link = av;
        av = temp;
        last = NULL;
    }

CircularList& CircularList::operator+(CircularList& lb) {
    Employee* p, * q;
    ListIterator Aiter(*this); ListIterator Biter(lb);
    CircularList lc;
    p = Aiter.First(); q = Biter.First();
    while (Aiter.NotNull() && Biter.NotNull()) {
        switch (p->compare(q)) {
            case '=':
                lc.Add(p);
                p = Aiter.Next(); q = Biter.Next();
                break;
            case '<':
                lc.Add(p);
                p = Aiter.Next();
                break;
            case '>':
                lc.Add(q);
                q = Biter.Next();
                break;
        }
    }
    while (Aiter.NotNull()) {
        lc.Add(p);
        p = Aiter.Next();
    }
    while (Biter.NotNull()) {
        lc.Add(q);
        q = Biter.Next();
    }
    return lc;
}

```

```

ListIterator::ListIterator(const CircularList& lst) : list(lst),
current(lst.last->link->link) {
}
bool ListIterator::NotNull() {
    if (current != list.last->link)
        return true;
    else
        return false;
}
bool ListIterator::NextNotNull() {
    if (current->link != list.last->link)
        return true;
    else
        return false;
}
Employee* ListIterator::First() {
    return &list.last->link->link->data;
}
Employee* ListIterator::Next() {
    current = current->link;
    Employee* e = &current->data;
    return e;
}

Employee* ListIterator::GetCurrent() {
    return &current->data;
}

ListIterator::~ListIterator() {
}

Employee& ListIterator::operator*() const {
    return current->data;
}

Employee* ListIterator::operator->()const {
    return &current->data;
}

```

```

}

ListIterator& ListIterator::operator++() {
    current = current->link;
    return *this;
}

ListIterator ListIterator::operator ++(int) {
    ListIterator old = *this;
    current = current->link;
    return old;
}

bool ListIterator::operator != (const ListIterator right) const {
    return current != right.current;
}

bool ListIterator::operator == (const ListIterator right) const {
    return current == right.current;
}

//int printAll(const List& l);//list iterator를 사용하여 작성하는 연습
//int sumProductFifthElement(const List& l);//list iterator를 사용하여 작성하는 연
습
int sum(const CircularList& l)
{
    ListIterator li(l);

    if (!li.NotNull()) return 0;
    Employee* emp = li.First();
    int retValue = emp->getSalary();
    while (li.NextNotNull() == true) {
        ++li;//current를 증가시킴
        //retvalue = retvalue + *li.Next();
        retValue = retValue + li.GetCurrent()->getSalary();//현재 current
가 가르키는 node의 값을 가져옴
    }
    return retValue;
}

```

```
}
```

```
double avg(const CircularList& l)
{
    ListIterator li(l);
    if (!li.NotNull()) return 0;
    int retvalue = li.First()->getSalary();
    int count = 1;
    while (li.NextNotNull() == true) {
        ++li;
        count++;
        //retvalue = retvalue + *li.Next();
        retvalue = retvalue + li.GetCurrent()->getSalary();
    }

    double result = (double)retvalue / count;

    return result;
}
```

```
int min(const CircularList& l)
{
    ListIterator li(l);
    if (!li.NotNull()) return -1;
    int minValue = li.First()->getSalary();

    while (li.NextNotNull() == true) {
        ++li;
        if (minValue > li.GetCurrent()->getSalary())
        {
            minValue = li.GetCurrent()->getSalary();
        }
    }
    return minValue;
}
```

```
int max(const CircularList& l)
```

```

{
    ListIterator li(l);
    if (!li.NotNull()) return -1;
    int maxValue = li.First()->getSalary();

    while (li.NextNotNull() == true) {
        ++li;
        if (maxValue < li.GetCurrent()->getSalary())
        {
            maxValue = li.GetCurrent()->getSalary();
        }
    }
    return maxValue;
}

enum Enum {
    Add0, Add1, Delete, Show, Search, Merge, SUM, AVG, MIN, MAX, Exit
};

Node* CircularList::av = NULL; //static 변수의 초기화 방법을 기억해야 한다

void main() {
    Enum menu: // 메뉴
    int selectMenu, num;
    string eno, ename;
    int pay;
    Employee* data;
    bool result = false;
    CircularList la, lb, lc;
    Employee* s;
    do {
        cout << endl << "0.Add0, 1.Add1, 2.Delete, 3.Show, 4.Search,
5.Merge, 6. sum, 7.avg, 8.min, 9.max, 10.exit 선택::";
        cin >> selectMenu;
        switch (static_cast<Enum>(selectMenu)) {
            case Add0:
                cout << "사원번호 입력:: ";

```

```

        cin >> eno;
        cout << "사원 이름 입력:: ";
        cin >> ename;
        cout << "사원 급여:: ";
        cin >> pay;
        data = new Employee(eno, ename, pay);
        la.Add(data);
        break;
case Add1:
    cout << "사원번호 입력:: ";
    cin >> eno;
    cout << "사원 이름 입력:: ";
    cin >> ename;
    cout << "사원 급여:: ";
    cin >> pay;
    data = new Employee(eno, ename, pay);
    lb.Add(data);
    break;
case Delete:
    cout << "사원번호 입력:: ";
    cin >> eno;
    result = la.Delete(eno);
    if (result)
        cout << "eno = " << eno << " 삭제 완료.";
    break;
case Show:
    cout << "리스트 la = ";
    la.Show();
    cout << "리스트 lb = ";
    lb.Show();
    break;
case Search:
    cout << "사원번호 입력:: ";
    cin >> eno;
    result = la.Search(eno);
    if (!result)
        cout << "검색 값 = " << eno << " 데이터가 없습니

```

```

다.";

else
    cout << "검색 값 = " << eno << " 데이터가 존재합
니다.";

break;
case Merge:
    lc = la + lb;
    cout << "리스트 lc = ";
    lc.Show();
    cout << "리스트 la를 삭제" << endl;
    la.Erase();
    cout << "리스트 lb를 삭제" << endl;
    lb.Erase();
    cout << "리스트 la = ";
    la.Show();
    cout << endl<<"리스트 lb = ";
    lb.Show();
    break;
case SUM: cout << "sum() = " << sum(la) << endl; break;
case AVG: cout << "avg() = " << avg(la) << endl; break;
case MIN: cout << "min() = " << min(la) << endl; break;
case MAX: cout << "max() = " << max(la) << endl; break;
case Exit: // 꼬리 노드 삭제
    break;
}
} while (static_cast<Enum>(selectMenu) != Exit);
cin >> num;
}

```

circular list에 head node를 추가하면 ~CircularChain()에서 first == 0인지를 검사할 필요가 없어진다. head node의 사용은 empty list 처리를 간단하게 할 수 있다. head node을 사용하여 empty list를 표현하면 addition과 deletion의 구현 알고리즘이 simple해진다. operator+() 처리 시에 잔여 노드들 copy하는 코드가 불필요해지기 때문에 알고리즘 코드가 간단해진다. 이것은 head node 와 같은 자료구조를 조금만 개선하여도 source code를 간단하게 구현할 수 있다는 것을 습득하는 것이 중요하다.

소스 코드 실습으로 head node 가 있는 circular list에 대한 iterator를 사용하여

리스트 merge 알고리즘을 구현한다. 이때 available list를 사용하는 것을 전제로 한다. operator+() 알고리즘[1]을 구현하는 실습 대상은 다음과 같다.

linked list 을 사용한 코딩 실습으로 1) head node가 없는 singly linked list의 operator+ ()의 구현, 2) head node가 있는 singly linked list의 operator+()의 구현, 3)head node가 있는 singly linked list에 대한 iterator를 사용한 operator+()의 구현, 4) 앞의 3가지에 대하여 circular list에 대한 3가지 반복에 대하여 코딩 실습을 하는데 avail list가 사용되고 template를 사용하는 전제조건으로 실습을 해야 한다.

4.6 Doubly linked list

singly linked lists에서 알고리즘 구현시 불편한 점은 현재 노드를 가리키는 변수 p의 preceding 노드가 필요한 경우이다. 임의 노드를 삭제할 때 preceding 노드를 알아야 한다. doubly linked list에서는 각 노드가 data 이외에 left link, right link를 포함하기 때문에 현재 노드의 앞의 노드를 가리키는 별도의 변수를 사용할 필요가 없다.

```
class DbList;
class DbListNode {
friend class DbList;
private:
    int data;
    DbListNode *llink, *rlink;
}
class DbList {
public:
    //list operations
private:
    DbListNode *first;
};
```

doubly linked list는 각 노드가 llink, rlink가 있으므로 head node가 필요하고 circular list가 되어야 llink와 rlink를 효과적으로 사용할 수 있다[1]. head

node가 있는 circular doubly link list에서 p가 한 노드를 가리킬 때 항상 다음 조건을 만족한다.

```
p == p->llink->rlink == p->rlink->llink
```

empty list는 head node만 있다. head node 만 있는 circular doubly linked list에서 Insert(), Delete() 함수로 노드의 추가 삭제 처리를 구현한 소스 코드를 사용하여 두개의 list의 merge 함수를 추가로 구현하여 본다.

//소스 코드4.7: Doubly Linked List

//circular doubly lists with head node 구현- 이 코드를 수정하여 사용, 두개의 list를 만들고 list node에 정수 값이 sorted된 상태로 입력되게 하고, 두개의 list를 merge하여 합친 결과도 sorted list되게 한다.

```
/*
 * doubly linked list
 6단계- 원형 객체 이중 연결 리스트의 available list, getNode, retNode
 head node를 갖고 있고 삭제된 노드들은 available list에 리턴한다.
 CircularDoublyList를 대상으로 한 iterator를 구현한다.
 */
#include <iostream>
#include <time.h>
using namespace std;
template<class T> class DoublyListNode;
template<class T> class CircularDoublyList;
template<class T> class CircularDoublyListIterator;
class Employee {
    template<class T> friend class DoublyListNode;
    template<class T> friend class CircularDoublyList;
    template<class T> friend class CircularDoublyListIterator;
private:
    string eno;
```

```

        string ename;
        int salary;
public:
    Employee() {}
    Employee(string sno, string sname, int salary) :eno(sno),
ename(sname), salary(salary) {}
    friend ostream& operator<<(ostream& os, Employee&);
    bool operator<(Employee&);
    bool operator>(Employee&);
    bool operator==(Employee&);
    char compare(const Employee* emp) const;
    int getSalary() const {
        return salary;
    }
};
ostream& operator<<(ostream& os, Employee& emp) {
    os << "<" << emp.eno << ", " << emp.ename << ", " << emp.salary <<
">";
    return os;
}
bool Employee::operator==(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename == emp.ename);
    else
        return false;
}
bool Employee::operator<(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename < emp.ename);
    else
        return (eno < emp.eno);
}
bool Employee::operator>(Employee& emp) {

```

```

        bool result = false;
        if (eno == emp.eno)
            return (ename > emp.eno);
        else
            return (eno > emp.eno);
    }
    char Employee::compare(const Employee* emp) const {
        if (this->eno > emp->eno)
            return '>';
        else if (this->eno < emp->eno)
            return '<';
        else
            return '=';
    }
}

template<class T> class CircularDoublyList;
template<class T> class CircularDoublyListIterator;
template<class T>
class DoublyListNode {
    friend class CircularDoublyList<T>;
    friend class CircularDoublyListIterator<T>;
public:
    DoublyListNode(T* p) {
        data = *p; llink = rlink = this;
    }
    DoublyListNode() {
        llink = rlink = this;
    }
private:
    T data;
    DoublyListNode<T>* llink, * rlink;
};

template<class T>
class CircularDoublyList {
    friend class CircularDoublyListIterator<T>;

```

```

public:
    CircularDoublyList() { last = new DoublyListNode<T>; }
    template<class T>
    friend ostream& operator<<(ostream&, CircularDoublyList<T>&);
    bool Delete(string);
    void Show();
    void Add(T*); //sno로 정렬되도록 구현
    bool Search(string);
    DoublyListNode<T> * GetNode();
    void RetNode(DoublyListNode<T> *);
    void Erase();
    CircularDoublyList<T>& operator+(CircularDoublyList<T>&);
private:
    DoublyListNode<T>* last;
    static DoublyListNode<T>* av;
};

template<class T>
class CircularDoublyListIterator {
public:
    CircularDoublyListIterator(const CircularDoublyList<T>& l) : list(l)
    {
        if (l.last == NULL) {
            cout << "리스트가 없다" <<endl;
        }
        else {
            current = l.last->rlink->rlink;
        }
    }
    ~CircularDoublyListIterator();
    bool NotNull();
    bool NextNotNull();
    T* First();
    T* Next();

```

```

        T* GetCurrent();
private:
        const CircularDoublyList<T>& list;
        DoublyListNode<T>* current;
};
template<class T>
DoublyListNode<T>* CircularDoublyList<T>::GetNode()
{ //provide a node for use
        DoublyListNode<T>* x;
        if (av) { x = av, av = av->rlink; }
        else x = new DoublyListNode<T>;
        return x;
}
template<class T>
void CircularDoublyList<T>::RetNode(DoublyListNode<T>* x)
{ //free the node pointed to by x
        x->rlink = av;
        av = x;
        x = 0;
}
template<class T>
void CircularDoublyList<T>::Show() { // 전체 리스트를 순서대로 출력한다.
        if (last == NULL)
                return;
        DoublyListNode<T>* first = last->rlink;
        DoublyListNode<T>* p = first->rlink;
        while (p != first) {
                cout << p->data;
                if (p != last)
                        cout << " -> ";
                p = p->rlink;
        }
        cout << endl;
}

```

```

template<class T>
void CircularDoublyList<T>::Add(T* element) // 임의 값을 삽입할 때 리스트가
오름차순으로 정렬이 되도록 한다
{
    DoublyListNode<T>* newNode = GetNode(); newNode->data =
*element;
    DoublyListNode<T>* first = last->rlink;
    DoublyListNode<T>* p = first->rlink;
    //Node* q = first;
    if (p == first) // insert into empty list
    {
        p->rlink = newNode; p->llink = newNode;
        newNode->llink = first; newNode->rlink = first;
        last = newNode;
        return;
    }
    // 기존 노드들이 있으면 순서가 유지하도록 삽입할 노드 위치를 찾는다

    while (p != first) {
        if (*element > p->data) { //operator<() 구현 필요
            p = p->rlink;
            if (p == first) {
                newNode->llink = p->llink; newNode->rlink =
p;

                p->llink->rlink = newNode;
                p->llink = newNode;
                last = newNode;
                return;
            }
        }
    }
    else {
        newNode->rlink = p; newNode->llink = p->llink;
        p->llink->rlink = newNode; p->llink = newNode;
        return;
    }
}

```

```

    }
}

template<class T>
bool CircularDoublyList<T>::Search(string eno) { // sno를 갖는 레코드를 찾기
    DoublyListNode<T>* first = last->rlink;
    DoublyListNode<T>* p = first->rlink;
    while (p != first) {
        if (p->data.eno == eno)
            return true;
        p = p->rlink;
    }
    return false;
}

template<class T>
bool CircularDoublyList<T>::Delete(string eno) // delete the element
{
    DoublyListNode<T>* first = last->rlink;
    DoublyListNode<T>* p = first->rlink;
    while (p != first) {
        if (p->data.eno == eno) // 삭제
        {
            if (p == last) {
                first->llink = p->llink;
                p->llink->rlink = first;
                last = p->llink;
                RetNode(p);
                return true;
            }
            p->rlink->llink = p->llink;
            p->llink->rlink = p->rlink;
            RetNode(p);
            return true;
        }
        else {

```

```

        p = p->rlink;
    }

}

return false;// 삭제할 대상이 없다.
}

template<class T>
void CircularDoublyList<T>::Erase() {
    DoublyListNode<T>* temp = last->rlink;
    last->rlink = av;
    av = temp;
    last = NULL;
}

template<class T>
ostream& operator<<(ostream& os, CircularDoublyList<T>& l)
{
    CircularDoublyListIterator<T> li(l);
    if (!li.NotNull()) return os;
    os << *li.First();
    while (li.NextNotNull())
        os << " + " << *li.Next();
    os << endl;
    return os;
}

template<class T>
C i r c u l a r D o u b l y L i s t < T > &
CircularDoublyList<T>::operator+(CircularDoublyList<T>& lb) {
    T* p, * q;
    CircularDoublyListIterator<T> Aiter(*this);
    CircularDoublyListIterator<T> Biter(lb);
    CircularDoublyList<T> lc;
    p = Aiter.First(); q = Biter.First();
    while (Aiter.NotNull() && Biter.NotNull()) {
        switch (p->compare(q)) {

```



```

        case '=':
            lc.Add(p);
            p = Aiter.Next(); q = Biter.Next();
            break;
        case '<':
            lc.Add(p);
            p = Aiter.Next();
            break;
        case '>':
            lc.Add(q);
            q = Biter.Next();
            break;
    }
}
while (Aiter.NotNull()) {
    lc.Add(p);
    p = Aiter.Next();
}
while (Biter.NotNull()) {
    lc.Add(q);
    q = Biter.Next();
}
return lc;
}

```

```

template<class T>
bool CircularDoublyListIterator<T>::NotNull() {
    if (current != list.last->rlink)
        return true;
    else
        return false;
}
template<class T>
bool CircularDoublyListIterator<T>::NextNotNull() {

```

```

        if (current->rlink != list.last->rlink)
            return true;
        else
            return false;
    }
    template<class T>
    T* CircularDoublyListIterator<T>::First() {
        return &list.last->rlink->rlink->data;
    }
    template<class T>
    T* CircularDoublyListIterator<T>::Next() {
        current = current->rlink;
        T* e = &current->data;
        return e;
    }
    template<class T>
    T* CircularDoublyListIterator<T>::GetCurrent() {
        return &current->data;
    }
    template<class T>
    CircularDoublyListIterator<T>::~CircularDoublyListIterator() {
    }

//int printAll(const List& l);//list iterator를 사용하여 작성하는 연습
//int sumProductFifthElement(const List& l);//list iterator를 사용하여 작성하는
연습
    template<class T>
    int sum(const CircularDoublyList<T>& l)
    {
        CircularDoublyListIterator<T> li(l);

        if (!li.NotNull()) return 0;
        Employee* emp = li.First();
        int retValue = emp->getSalary();
        while (li.NextNotNull() == true) {

```

```

        li.Next();
        retValue += retValue + li.GetCurrent()->getSalary();//현재
current가 가르키는 node의 값을 가져옴
    }
    return retValue;
}
template<class T>
double avg(const CircularDoublyList<T>& l)
{
    CircularDoublyListIterator<T> li(l);
    if (!li.NotNull()) return 0;
    int retvalue = li.First()->getSalary();
    int count = 1;
    while (li.NextNotNull() == true) {
        count++;
        li.Next();
        retvalue += li.GetCurrent()->getSalary();
    }

    double result = (double)retvalue / count;

    return result;
}
template<class T>
int min(const CircularDoublyList<T>& l)
{
    CircularDoublyListIterator<T> li(l);
    if (!li.NotNull()) return -1;
    int minValue = li.First()->getSalary();

    while (li.NextNotNull() == true) {
        li.Next();
        if (minValue > li.GetCurrent()->getSalary())
        {
            minValue = li.GetCurrent()->getSalary();
        }
    }
}

```

```

        }
    }
    return minValue;
}

template<class T>
int max(const CircularDoublyList<T>& l)
{
    CircularDoublyListIterator<T> li(l);
    if (!li.NotNull()) return -1;
    int maxValue = li.First()->getSalary();

    while (li.NextNotNull() == true) {
        li.Next();
        if (maxValue < li.GetCurrent()->getSalary())
        {
            maxValue = li.GetCurrent()->getSalary();
        }
    }
    return maxValue;
}

enum Enum {
    Add0, Add1, Delete, Show, Search, Merge, SUM, AVG, MIN, MAX,
    Exit
};

template<class T>
DoublyListNode<T>* CircularDoublyList<T>::av = 0;
/*
Node* CircularDoublyList::av = NULL;//static 변수의 초기화 방법을 기억해야 한다
*/
void main() {
    Enum menu; // 메뉴
    int selectMenu, num;
    string eno, ename;

```

```

int pay;
Employee* data;
bool result = false;
CircularDoublyList<Employee> la, lb, lc;
Employee* s;
do {
    cout << endl << "0.Add0, 1.Add1, 2.Delete, 3.Show, 4.Search,
5.Merge, 6. sum, 7.avg, 8.min, 9.max, 10.exit 선택::";
    cin >> selectMenu;
    switch (static_cast<Enum>(selectMenu)) {
    case Add0:
        cout << "사원번호 입력:: ";
        cin >> eno;
        cout << "사원 이름 입력:: ";
        cin >> ename;
        cout << "사원 급여:: ";
        cin >> pay;
        data = new Employee(eno, ename, pay);
        cout << *data;
        la.Add(data);
        break;
    case Add1:
        cout << "사원번호 입력:: ";
        cin >> eno;
        cout << "사원 이름 입력:: ";
        cin >> ename;
        cout << "사원 급여:: ";
        cin >> pay;
        data = new Employee(eno, ename, pay);
        cout << *data;
        lb.Add(data);
        break;
    case Delete:
        cout << "사원번호 입력:: ";
        cin >> eno;

```

```

        result = la.Delete(eno);
        if (result)
            cout << "eno = " << eno << " 삭제 완료.";
        break;
case Show:
    cout << "리스트 la = ";
    la.Show();
    cout << "리스트 lb = ";
    lb.Show();
    break;
case Search:
    cout << "사원번호 입력:: ";
    cin >> eno;
    result = la.Search(eno);
    if (!result)
        cout << "검색 값 = " << eno << " 데이터가 없
습니다.";

    else
        cout << "검색 값 = " << eno << " 데이터가 존
재합니다.";

    break;
case Merge:
    lc = la + lb;
    cout << "리스트 lc = ";
    lc.Show();
    cout << "리스트 la를 삭제" << endl;
    la.Erase();
    cout << "리스트 lb를 삭제" << endl;
    lb.Erase();
    cout << "리스트 la = ";
    la.Show();
    cout << endl << "리스트 lb = ";
    lb.Show();
    break;
case SUM:  cout << "sum() = " << sum(la) << endl; break;

```

```

        case AVG:  cout << "avg() = " << avg(la) << endl; break;
        case MIN:  cout << "min() = " << min(la) << endl; break;
        case MAX:  cout << "max() = " << max(la) << endl; break;
        case Exit: // 꼬리 노드 삭제
                    break;
    }
} while (static_cast<Enum>(selectMenu) != Exit);
cin >> num;
}

```

4.7 Linked list를 사용한 Polynomial 처리

Polynomial을 linked list class를 사용하여 구현한다. linked list를 사용하여 polynomial을 구현하면 polynomials 항들은 노드로 얼마든지 표현 가능하다. polynomials 을 구현하기 위해 Polynomial class를 정의할 때 Polynomial의 data member로서 LinkedList로 선언된 변수 poly를 사용한다. Polynomial의 data member인 *poly*는 linked list object를 갖게 된다[1].

4.7절에서는 LinkedList 대신에 Class Chain 명칭을 사용하고 class Node 대신에 class ChainNode 명칭을 사용한다. 그리고 polynomial의 각 항을 표현하기 위하여 class Term을 정의한다. class Term은 template<class T>로 public data member로서 coef와 exp를 포함한다. coef, exp를 public으로 선언하는 이유는 template<class T> class Polynomial에서 Chain<Term<T> > poly;을 사용하기 위함이다. class Term, class ChainNode, class Chain, class ChainIterator, class Polynomial 관계를 잘 이해하는 것이 필요하다.

두 개의 polynomial을 add하는 Polynomial<T> Polynomial<T>:: operator+(const Polynomial<T>& b) const { }의 body에서 merge한 후에 남아 있는 노드를 처리하기 위한 while문이 필요하다. 추가로 소스 코드를 개발하기 위한 실습 과제로 $f(x) = 2x^5 + 3x^4 + 7$ 에 대하여 $f(1.33)$ 의 값을 eval() 함수를 만든다. 그리고 다항식의 곱셈 $f(x) * g(x)$ 로서 모든 항끼리 곱한 다음에 지수 순서로 내림차순으로 circular list를 만드는 실습을 한다.

다항식 덧셈을 처리하는 main() 함수에서 Polynomial<int> a, b, sum;를 호출하면 polynomial class에서 Chain<Term<int>> poly;가 된다. Chain class에서 ChainNode<T>* first;은 ChainNode<Term<int>>* first;이 된다. ChainNode class에서 T data; ChainNode<T>* link;은 Term<int> data; ChainNode<Term<int>>* link;이 된다. Term class 내부의 {T coef; T exp; }은 {int coef; int exp;}로 처리된다. template를 사용한 코딩실습을 해야 한다.

```
//소스 코드4.7: Polynomial Linked List
```

```
/*소스 코드4.7: Polynomial Linked List
```

```
//지수는 정수 0<= 지수 <= 5, 난수로 생성한다
```

```
//계수는 double로 난수 생성한다. -9.0 < 계수 < 9.0
```

```
- 지수 내림 차순으로 정렬한다.
```

```
- c = a.Add(b) 구현한다.
```


- singly linked list, circular linked list, circular linked list with head node

1) head node가 있는 원형 리스트

2) Chainiterator를 사용한 template version으로 구현

3) 다항식 입력은 화면 입력 또는 난수 생성하여 자동 입력 무방

```
*/
#include <iostream>
#include<string>
#include <assert.h>
using namespace std;

template<class T> class Term {
public:
    //All members of Term are public by default
    T coef; //coefficient
    int exp; //exponent
    Term() { coef = 0; exp = 0; }
    Term(T c, T e) :coef(c), exp(e) { }
    Term Set(int c, int e) { coef = c; exp = e; return *this; };
};

template<class T> class Chain; //forward declaration
template<class T> class ChainIterator;

template<class T>
class ChainNode {
    friend class Chain<T>;
    friend class ChainIterator<T>;
public:
    ChainNode() { }
    ChainNode(const T&);
private:
    T data;
    ChainNode<T>* link;
};

template<class T> class Chain {
public:
```

```

Chain() { first = 0; };
void Delete(void); //delete the first element after first
int Length();
void Add(const T& element); //add a new node after first
void Invert();
void Concatenate(Chain<T> b);
void InsertBack(const T& element);
void displayAll();

ChainIterator<T> begin() const { return ChainIterator<T>(first); }
ChainIterator<T> end() const { return ChainIterator<T>(nullptr); }
private:
    ChainNode<T>* first;
};

template<class T> class ChainIterator {
private:
    //const Chain<T>* list; //refers to an existing list
    ChainNode<T>* current; //points to a node in list
public:
    //ChainIterator<T>(const Chain<T> &l) :Chain(l), current(l.first) { };
    ChainIterator() { }
    ChainIterator(ChainNode<T>* node) :current(node) { }
    //dereferencing operators
    T& operator *() const { return current->data; }
    T* operator ->()const { return &current->data; }
    bool operator && (ChainIterator<T> iter)const {
        return current != NULL && iter.current != NULL;
    }
    bool isEmpty() const { return current == NULL; }
    //increment
    ChainIterator& operator ++(); //preincrement
    ChainIterator<T> operator ++(int); //post increment
    bool NotNull(); //check the current element in list is non-null
    bool NextNotNull(); //check the next element in list is non-null
    //T* First( ) { //return a pointer to the first element of list
    //      if (list->first) return &list->first->data;

```

```

        //      else return 0;
    //}
    T* Next();//return a pointer to the next element of list
};

/*
class Polynomial
*/
template<class T> class Polynomial {
public:
    Polynomial() { }
    Polynomial(Chain<Term<T> >* terms) :poly(terms) { }
    Polynomial<T> operator+(const Polynomial<T>& b) const;
    void add(T coef, T exponent);
    void addAll(Polynomial<T>* poly);
    void display();
    /*
    T Evaluate(T&) const;//f(x)에 대하여 x에 대한 값을 구한다
    polynomial<T> Multiply(Polynomial<T>&); //f(x) * g(x)
    Polynomial(const Polynomial& p); //copy constructor
    friend istream& operator>>(istream&, Polynomial&);//polynomial 입력
    friend ostream& operator<<(ostream&, Polynomial&);//polynomial 출력
    const Polynomial& operator=(const Polynomial&) const;
    ~Polynomial( );
    Polynomial operator-(const Polynomial&)const;
    */
private:
    Chain<Term<T> > poly;
};

template <typename valType>
inline ostream& operator<< (ostream& os, const Term<valType>& term) {
    os << term.coef << "^" << term.exp;
    return os;
}

template <class T>

```

```
ChainNode<T>::ChainNode(const T& element) {
    data = element;
    link = 0;
}
```

```
template <class T>
void Chain<T>::Delete(void) //delete the first element after first
{
    ChainNode<T>* current, * next;
    next = first->link;
    if (first != nullptr) //non-empty list
    {
        ChainNode<T>* temp = first;
        first = next;
        delete temp;
    }
    else cout << "Empty List: Not deleted" << endl;
}
```

```
template <class T>
void Chain<T>::Add(const T& element) //add a new node after first
{
    ChainNode<T>* newNode = new ChainNode<T>(element);
    if (first == nullptr) // insert into empty list
    {
        first = newNode;
        return;
    }
    // 기존 노드들이 있으면 순서가 유지하도록 삽입할 노드 위치를 찾는다
    ChainNode<T>* p = first, * q = nullptr;
    while (p != nullptr) {
        if (p->data.exp > element.exp) {
            q = p;
            p = p->link;
            if (p == nullptr) {
                q->link = newNode;
                return;
            }
        }
    }
    q->link = newNode;
    return;
}
```

```

        }
    }
    else {
        newNode->link = p;
        if (q != nullptr)
            q->link = newNode;
        else
            first = newNode;
        return;
    }
}

template <class T>
void Chain<T>::Invert() {
    ChainNode<T>* p = first, * q = 0; //q trails p
    while (p) {
        ChainNode<T>* r = q; q = p; //r trails q
        p = p->link; //p moves to next node
        q->link = r; //link q to preceding node
    }
    first = q;
}

template <class T>
void Chain<T>::Concatenate(Chain<T> b) {
    if (!first) { first = b.first; return; }
    if (b.first) {
        for (ChainNode<T>* p = first; p->link; p = p->link) {
            p->link = b.first;
        }
    }
}

template <class T>
void Chain<T>::InsertBack(const T& element) {
    ChainNode<T>* newnode = new ChainNode<T>(element);

```

```

    if (!first) //insert into empty list
    {
        first = newnode;
        return;
    }
    ChainNode<T>* curr = first;
    while (curr->link != NULL) {
        curr = curr->link;
    }
    curr->link = newnode;
}

```

```

template <class T>
void Chain<T>::displayAll() {
    if (!first) return;
    ChainNode<T>* curr = first;
    while (curr != NULL) {
        cout << curr->data << " + ";
        curr = curr->link;
    }
    cout << endl;
}

```

```

template <class T>
ChainIterator<T>& ChainIterator<T>::operator ++() //preincrement
{
    current = current->link;
    return *this;
}

```

```

template <class T>
ChainIterator<T> ChainIterator<T>::operator ++(int) //post increment
{
    ChainIterator<T> old = *this;
    current = current->link;
    return old;
}

```

```

template <class T>
bool ChainIterator<T>::NotNull() { //check the current element in list is non-null
    if (current) return 1;
    else return 0;
}

template <class T>
bool ChainIterator<T>::NextNotNull() { //check the next element in list is
non-null
    if (current && current->link) return 1;
    else return 0;
}

template <class T>
T* ChainIterator<T>::Next() { //return a pointer to the next element of list
    if (current) {
        current = current->link;
        return &current->data;
    }
    else return 0;
}

template<class T>
void Polynomial<T>::add(T coef, T exponent) {
    Term<T>* newTerm = new Term<T>(coef, exponent);
    this->poly.Add(*newTerm);
}

template<class T> void Polynomial<T>::addAll(Polynomial<T>* b) {
    ChainIterator<Term<T>> iterB = b->poly.begin();

    while (iterB.NotNull()) {
        Term<T> dataB = *iterB;
        add(dataB.coef, dataB.exp);
        iterB.Next();
    }
}

```

```

}

template<class T> void Polynomial<T>::display() {
    poly.displayAll();
    cout << endl;
}

template<class T>
Polynomial<T> Polynomial<T>::operator+(const Polynomial<T>& b) const {
    Term<T> temp;
    ChainIterator<Term<T>> ai = poly.begin(), bi = b.poly.begin();
    Polynomial<T> c;

    while (ai && bi) { //current nodes are not null

        if (ai->exp == bi->exp) {
            int sum = ai->coef + bi->coef;
            if (sum) c.poly.InsertBack(temp.Set(sum, ai->exp));
            ai++, bi++; //advance to next term
        }
        else if (ai->exp < bi->exp) {
            c.poly.InsertBack(temp.Set(bi->coef, bi->exp));
            bi++; //next term of b
        }
    }

    while (!ai.isEmpty()) { //copy rest of a
        c.poly.InsertBack(temp.Set(ai->coef, ai->exp));
        ai++;
    }
    while (!bi.isEmpty()) { //copy rest of b
        c.poly.InsertBack(temp.Set(bi->coef, bi->exp));
        bi++;
    }
    return c;
}

```



```

int main(void) {

    Polynomial<int> a, b, sum;

    char select;
    Term<int>* tempTerm;
    ChainNode<Term<int>> cn;
    Chain<Term<int>> ca, cb;
    ChainIterator<Term<int>> iter;
    int c, e;

    cout << endl << "명령 선택: a: Add_a, b: Add_b, p: a + b, d: DisplayAll,
q: exit" << endl;
    cin >> select;
    while (select != 'q')
    {
        switch (select)
        {
            case 'a':
                cout << "리스트 a의 항 입력 : " << endl;
                cout << "정수 coef: ";
                cin >> c;
                cout << "정수(내림차순) exp: ";
                cin >> e;
                a.add(c, e); //지수를 임의 숫자로 입력하여도 지수 내림 차순
                               으로 정렬
                break;
            case 'b':
                cout << "리스트 b의 항 입력: " << endl;
                cout << "정수 coef: ";
                cin >> c;
                cout << "정수(내림차순) exp: ";
                cin >> e;
                b.add(c, e);
                break;
            case 'p': //a+b

```

```

        cout << "a+b: ";
        //a.addAll(&b);
        a.display();
        b.display();
        sum = a + b;
        sum.display();
        //cout << sum;
        break;
    case 'd':
        cout << "display all: " << endl;
        a.display();
        b.display();
        break;
    default:
        cout << "WRONG INPUT " << endl;
        cout << "Re-Enter" << endl;
    }
    cout << endl << "Select command: a: Add_a, b: Add_b, p: Plus,
d: DisplayAll, q: exit" << endl;
    cin >> select;
}
system("pause");
return 0;
}

```

4.8 Available list를 사용한 Polynomial 처리

$d(x) = a(x) + b(x) + c(x)$ 를 계산하기 위해 $t = a + b$ 를 먼저 구한 후에 $d = t + c$ 를 처리한다. 함수 `func()`는 다항식 a, b, c, d, t 를 생성하고 a, b, c 를 입력받는다.

```
viod func( )
{
    polynomial a, b, c, d, t;
    cin >> a; // read and create polynomials
    cin >> b;
    cin >> c;
    t = a + b
    d = t + c;
    cout << d;
}
```

다항식 연산을 처리하는 `func` 이 종료되면 모든 list의 polynomial 객체들은 반환되어야 한다. 즉, 다항식 객체 a, b, c, t, d 에 사용된 모든 노드들이 반환되어야 한다. 그런데 함수 `func`이 종료되면 polynomial 객체의 destructor가 호출되므로 polynomial 객체들은 없어지나 data member가 가리키는 노드들은 삭제되지 않는다. 즉, `new`에 의해 생성된 노드들은 `delete`에 의해 삭제되어야 한다. 따라서 polynomial 객체들만 삭제하면 노드들은 더 이상 접근이 가능하지 않은 dangling reference가 발생하게 된다.

`class List`의 destructor는 dangling reference가 발생하지 않도록 하기 위해 모든 노드를 방문하여 `delete`하는 loop가 필요하게 된다.

```
template <class T>
List<T>::~~List( ) ( ) {
    ListNode<T> *next;
    for (; first; first = next) {
        next = first->link;
    }
}
```

```

        delete first;
    }
}

```

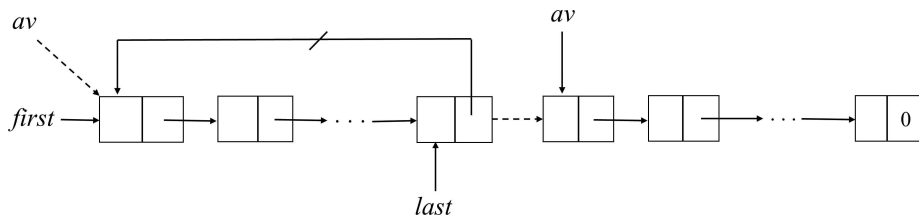


그림 4.6 Available List.

circular list를 이용하여 list의 모든 노드를 반환하는 것을 간단하게 구현할 수 있다. circular lists를 위한 erase algorithm을 간단하게 구현하기 위해서 삭제된 ChainNode들을 *av* list, 즉 available-space list[1]로 유지한다. *av* list가 empty 이면 새로운 ChainNode를 만들기 위해 *new* 을 사용한다. *av*는 `CircList<Type>` 의 static class member로 선언한다.

//소스 코드4.5: Available Linked List

// circular list를 사용한 version, available list를 사용한 구현

```

#include <iostream>
#include <time.h>
#include <cstdlib>
using namespace std;

template <class Type> class CircularList;
template <class Type> class CircularListIterator;

template<class T> class Term {
public:
    //All members of Term are public by default
    T coef; //coefficient
    int exp; //exponent
    Term() { coef = 0; exp = 0; }
    Term(T c, T e) :coef(c), exp(e) { }

```

```

    void NewTerm(float coef, int exp) {
        this->coef = coef; this->exp = exp;
    }
    template <class T>
    friend ostream& operator<<(ostream& os, Term<T>& e);
};

template<class T>
ostream& operator<<(ostream& os, Term<T>& e)
{
    os << e.coef << "X**" << e.exp;
    return os;
}

template <class Type>
class ListNode {
    friend class CircularList<Type>;
    friend class CircularListIterator<Type>;
private:
    Type data;
    ListNode<Type>* link;
public:
    ListNode(Type);
    ListNode();
    template <class Type>
    friend ostream& operator<<(ostream&, ListNode<Type>&);
};

template <class Type>
ostream& operator<<(ostream& os, ListNode<Type>& ln) {
    os << ln.data
    return os;;
}

template <class Type>
ListNode<Type>::ListNode(Type Default)
{
    data = Default;
    link = 0;
}

```

```

}
template <class Type>
ListNode<Type>::ListNode()
{
    data = * new Type;
    link = 0;
}

```

```

template <class Type>
class CircularList {
    friend class CircularListIterator<Type>;
public:
    CircularList() { last = new ListNode<Type>; last->link = last;};
    CircularList(const CircularList&);
    ~CircularList();
    void Attach(Type);
    ListNode<Type>* GetNode();
    void RetNode(ListNode<Type>*);
    void Erase();
    template <class Type>
    friend ostream& operator<< (ostream& os, CircularList<Type>& cl);
private:
    ListNode<Type>* last;
    static ListNode<Type>* av;
};

```

```

template <class Type>
ListNode<Type>* CircularList<Type>::av = 0; //static 사용시 초기화 방법
//ListNode<Term> CircularList<Term>::av = 0; //오류 발생

```

```

template <class Type>
class CircularListIterator {
public:
    CircularListIterator(const CircularList<Type>& l) : list(l) { current =
l.last->link; }
    Type* Next();
    Type* First();

```

```

        bool NotNull();
        bool NextNotNull();
private:
        const CircularList<Type>& list;
        ListNode<Type>* current;
};

template <class Type>
Type* CircularListIterator<Type>::First() {
    if (current->link != list.last->link) {
        current = current->link;
        return &current->data;
    }
    else return 0;
}

template <class Type>
Type* CircularListIterator<Type>::Next() {
    current = current->link;
    if (current != list.last->link) return &current->data;
    else return 0;
}

template <class Type>
bool CircularListIterator<Type>::NotNull()
{
    //if (current == nullptr)
    //    return false;
    if (current->link != list.last->link) return true;
    else return false;
}

template <class Type>
bool CircularListIterator<Type>::NextNotNull()
{
    if (current != list.last->link) return true;
    else return false;
}

```

```
}
```

```
template <class Type>
```

```
CircularList<Type>::CircularList(const CircularList<Type>& l)
```

```
{
```

```
    if (l.last->link == l.last) { last = new ListNode<Type>; return; }
```

```
    ListNode<Type>* first = l.last->link;
```

```
    ListNode<Type>* p = new ListNode<Type>;//head node
```

```
    p->link = p;
```

```
    last = p;
```

```
    for (ListNode<Type>* current = first->link; current != first; current =  
current->link)
```

```
    {
```

```
        ListNode<Type>* temp = new ListNode<Type>(current->data);
```

```
        p->link = temp; p = temp;
```

```
    }
```

```
    p->link = last;
```

```
    last = p;
```

```
}
```

```
template <class Type>
```

```
CircularList<Type>::~~CircularList()
```

```
{
```

```
    if (last)
```

```
    {
```

```
        ListNode<Type>* tmp = last->link;
```

```
        last->link = av;
```

```
        av = tmp;
```

```
    }
```

```
}
```

```
template <class Type>
```

```
ostream& operator<<(ostream& os, CircularList<Type>& l)
```

```
{
```

```
    cout << "원형 리스트 출력 " << endl;
```

```
    CircularListIterator<Type> li(l);
```



```

        if (!li.NotNull()) return os;
        os << *li.First();
        while (li.NotNull())
            os << " + " << *li.Next();
        os << endl;
        return os;
    }

template <class Type>
void CircularList<Type>::Attach(Type k)
{
    ListNode<Type>* p = last;
    ListNode<Type>* newnode = new ListNode<Type>(k);
    if (p->link == last) {
        p->link = newnode;
        newnode->link = last;
        last = newnode;
    }
    else {
        newnode->link = last->link;
        last->link = newnode;
        last = newnode;
    }
}

template <class Type>
ListNode<Type>* CircularList<Type>::GetNode()
{ //provide a node for use
    ListNode<Type>* x;
    if (av) { x = av, av = av->link; }
    else x = new ListNode<Type>;
    return x;
}

template <class Type>
void CircularList<Type>::RetNode(ListNode<Type>* x)
{ //free the node pointed to by x
    x->link = av;
    av = x;
}

```

```

}
template <class Type>
void CircularList<Type>::Erase() {
    ListNode<Type>* temp = last->link;
    last->link = av;
    av = temp;
    last = NULL;

}
//////////Polynomial//////////
template <class Type>
class Polynomial
{
private:
    CircularList<Term<Type>> poly;
public:
    Polynomial(CircularList<Term<Type> >* terms) :poly(terms) { }
    template <class Type>
    friend ostream& operator<<(ostream&, Polynomial<Type>&);//polynomial
출력
    /*
    template <class Type>
    friend istream& operator>>(istream&, Polynomial&);//polynomial 출력
    */
    Polynomial<Type>* operator+(const Polynomial<Type>
&)const;//polynomial ADD( )
    Polynomial();//생성자 정의 필요-head node를 갖는 경우
    void Add(Term<Type> e);
    int GetData();
    void Erase();
    /*
    T Evaluate(T&) const;//f(x)에 대하여 x에 대한 값을 구한다
    Polynomial<T> Multiply(Polynomial<T>&); //f(x) * g(x)
    Polynomial(const Polynomial<T>& p); //copy constructor
    friend istream& operator>>(istream&, Polynomial<T>&);//polynomial 입력
    friend ostream& operator<<(ostream&, Polynomial<T>&);//polynomial 출
력

```

```

        const Polynomial<T>& operator=(const Polynomial<T>&) const:
        ~Polynomial( );
        Polynomial<T> operator-(const Polynomial<T>&)const:
        */
};

template <class Type>
Polynomial<Type>::Polynomial()
{
    poly = *new CircularList<Term<Type>>;
}

template <class Type>
int Polynomial<Type>::GetData() {
    int i, degree;
    float coef;
    int expo;
    int maxExp = 999;

    do {
        Term<float> m;
        i = rand() % 10;
        if (i == 0)
            continue;
        coef = (float)i / 10.0;
        expo = rand() % 9;
        if (expo >= maxExp)
            continue;
        maxExp = expo;
        m.NewTerm(coef, expo);
        poly.Attach(m);
    } while (maxExp > 0);
    return 0;
}

template <class Type>
void Polynomial<Type>::Add(Term<Type> e)
{
    poly.Attach(e);
}

```

```

}
template <class Type>
void Polynomial<Type>::Erase() {
    poly.Erase();
}

template <class Type>
ostream& operator<<(ostream& os, Polynomial<Type>& p)
{
    os << p.poly;
    return os;
}

char compare(int a, int b)
{
    if (a == b) return '=';
    if (a < b) return '<';
    return '>';
}

/*
CircularListIterator<Type> li(l);
if (!li.NotNull()) return os;
os << *li.First();
while (li.NextNotNull())
os << " + " << *li.Next();
*/
template <class Type>
Polynomial<Type>* Polynomial<Type>::operator+(const Polynomial<Type>& b)const
{
    Term<Type>* p, * q, *temp;
    CircularListIterator<Term<Type>> Aiter(poly);
    CircularListIterator<Term<Type>> Biter(b.poly);
    Polynomial<Type> *c = new Polynomial<Type>;

    p = Aiter.First();
    q = Biter.First();
    float x = 0.0;

```

```

while (Aiter.NextNotNull() && Biter.NextNotNull()) {
    Term<float> m;
    switch (compare(p->exp, q->exp)) {
    case '=':
        x = p->coef + q->coef;
        m.NewTerm(x, q->exp);
        if (x) c->poly.Attach(m);
        p = Aiter.Next();
        q = Biter.Next();
        break;
    case '<':
        m.NewTerm(q->coef, q->exp);
        c->poly.Attach(m);
        q = Biter.Next();
        break;
    case '>':
        m.NewTerm(p->coef, p->exp);
        c->poly.Attach(m);
        p = Aiter.Next();
    }
}
while (Aiter.NextNotNull()) {
    Term<float> m;
    m.NewTerm(p->coef, p->exp);
    c->poly.Attach(m);
    p = Aiter.Next();
}
while (Biter.NextNotNull()) {
    Term<float> m;
    m.NewTerm(q->coef, q->exp);
    c->poly.Attach(m);
    q = Biter.Next();
}
return c;
}
//adding two polynomials as circular lists with head nodes=> 수정하는 것을 실습

```

```

int main()
{
    srand(time(NULL));

    Polynomial<float> p, q, r, * s, * t;
    char select;
    Term<float> e;
    cout << endl << "Select command: a: 다항식 입력, b: p+q, c: (p+q)+r, q:
exit" << endl;
    cin >> select;
    while (select != 'q')
    {
        switch (select)
        {
            case 'a':
                p.GetData();
                q.GetData();
                r.GetData();
                cout << "다항식 p, q, r 입력 결과::";
                cout << p;
                cout << q;
                cout << r;
                break;
            case 'b': //a+b
                s = p + q;
                cout << "a = p + q 실행결과::";
                cout << p;
                cout << q;
                cout << *s;
                cout << "다항식 p, q를 삭제";
                p.Erase(); q.Erase();
                break;
            case 'c':
                t = *s + r;
                cout << "t = s + r 실행결과::";
                cout << *s;
                cout << r;
        }
    }
}

```

```

        cout << *t;
        cout << "다항식 s, r를 삭제";
        s->Erase(); r.Erase();
        break;
    default:
        cout << "WRONG INPUT " << endl;
        cout << "Re-Enter" << endl;
    }
    cout << endl << "Select command: a: 다항식 입력, b: p+q, c:
(p+q)+r, q: exit" << endl;
    cin >> select;
}
system("pause");
return 0;
}

```

available list를 사용함으로써 new, delete 대신에 CircularChain::GetNode와 Circular Chain::RetNode를 사용한다. CircularChains내에 static ChainNode<T>* av;를 class 변수로 기술한다.

circular list에 head node를 추가하면 ~CircularChain()에서 first == 0인지를 검사할 필요가 없어진다. head node의 사용은 zero 다항식 처리를 간단하게 할 수 있다. head node를 사용하여 empty polynomial을 표현하고 addition과 deletion의 구현 알고리즘이 simple해진다. 다항식을 표현하는 operator+() 처리 시에 잔여 terms을 copy하는 코드가 iterator 함수를 사용하여 간단하게 구현된다. 이것은 head node 와 같은 자료구조를 조금만 개선하여도 source code를 간단하게 구현할 수 있다는 것을 습득하는 것이 중요하다.

소스 코드 실습으로 head node 가 있는 circular list에 대한 iterator를 사용하여 다항식 덧셈 알고리즘을 구현한다. 이때 available list를 사용하는 것을 전제로 한다.

linked list 을 사용한 코딩 실습으로 1) head node가 없는 singly linked list의 operator+ ()의 구현, 2) head node가 있는 singly linked list의 operator+()의 구현, 3)head node가 있는 singly linked list에 대한 iterator를 사용한 operator+()의 구현, 4) 앞의 3가지에 대하여 circular list에 대한 3가지 반복에

대하여 코딩 실습을 하는데 avail list가 사용되고 template를 사용하는 전제조건
으로 실습을 해야 한다.



4.9 stack과 queue의 linked list 구현

복수 개의 stacks과 queues가 필요한 경우 stack과 queue의 데이터 멤버를 배열을 사용하여 표현하면 stack full, queue full 문제 때문에 add, delete 처리 시에 배열의 element를 shift해야 하는 문제가 발생한다. 만약 m개의 stack과 n개의 queue가 필요할 경우에 full이 된 stack과 queue의 용량을 2배로 늘리기 위해서는 ChangeSizeID()에서 memcpy()를 사용하여 새로 할당된 배열에 기존 데이터를 shift해야 한다. 즉, m stacks과 n queues를 사용하기 위해서 m stacks과 n queues의 내부 데이터 멤버를 배열로 구현하면 full이 되는 stack과 queue의 용량을 늘리는 함수를 구현해야 한다[1].

```
Stack *stack = new Stack[m];
```

```
Queue *queue = new Queue[n];
```

stack과 queue의 구현을 배열 대신에 linked list로 구현함으로써 동적인 메모리 할당 효과를 갖기 때문에 stack과 queue가 full이 되는 문제가 발생하지 않는다. class StackNode를 사용하여 linked list에 해당되는 class Stack을 정의한다. 마찬가지로 class QueueNode를 사용하여 linked list에 해당되는 class Queue를 만든다. linked list를 사용한 stack과 queue의 구현은 stack, queue의 멤버 데이터의 크기를 2배로 늘리는 함수 구현이 필요하지 않다.

//소스 코드4.9

```
/*
7단계: Linked List를 사용한 스택과 큐의 구현
*/

//Stack node & Queue Node
#include <stdio.h>
#include <iostream>
using namespace std;
class Employee {
    friend class Node;
    friend class LinkedList;
    string eno;
    string ename;
```

```

public:
    Employee() {}
    Employee(string sno, string sname) :eno(sno), ename(sname) {}
    friend ostream& operator<<(ostream& os, Employee&);
    bool operator<(Employee&);
    bool operator==(Employee&);
};

ostream& operator<<(ostream& os, Employee& emp) {
    os << " " << emp.eno << emp.ename;
    return os;
}

bool Employee::operator==(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename == emp.ename);
    else
        return false;
}

bool Employee::operator<(Employee& emp) {
    bool result = false;
    if (eno == emp.eno)
        return (ename < emp.ename);
    else
        return (eno < emp.eno);
}

template <class T> class Stack; //forward declaration
template <class T> class Queue;
template <class T>
class StackNode {
    friend class Stack<T>;
private:
    T* data;
    StackNode<T>* link;
public:
    StackNode<T>(T* d = 0) : data(d) { link = 0; } //constructor
};

```

```

template <class T>
class QueueNode {
    friend class Queue<T>;
private:
    T* data;
    QueueNode<T>* link;
public:
    QueueNode<T>(T* d = 0) : data(d) { link = 0; }//constructor
};

```

```

template <class T>
class Stack {
public:
    Stack<T>() { top = 0; }//constructor
    void Add(T*);
    T* Delete();
    void Show();
private:
    StackNode<T>* top;
    void StackEmpty();
};

```

```

template <class T>
class Queue {
public:
    Queue<T>() { front = rear = 0; }//constructor
    void Add(T*);
    T* Delete();
    void ShowQueue();
private:
    QueueNode<T>* front;
    QueueNode<T>* rear;
    void QueueEmpty();
};

```

```

template <class T>
void Stack<T>::Add(T* y) {
    StackNode<T>* temp = new StackNode<T>(y);
    if (top != NULL) {
        temp->link = top;
    }
    top = temp;
}

template <class T>
void Stack<T>::StackEmpty() {
    if (top == 0)
        cout << "Stack is empty" << endl;
    else
        cout << "Stack is non empty" << endl;
}

template <class T>
void Stack<T>::Show() {
    StackNode<T>* p = top;
    while (p != NULL) {
        cout << *p->data;
        if (p -> link != NULL)
            cout << " > ";
        p = p->link;
    }
    cout << endl;
}

template <class T>
T* Stack<T>::Delete()
//delete the top node from stack and return a pointer to its data
{
    if (top == 0) {
        StackEmpty(); return 0;
    } //return null pointer constant
    StackNode<T>* x = top;
    Employee *retvalue = top->data; //get data field of top node
    top = x->link; //remove top node
}

```

```

        delete x;
        return retvalue;//return pointer to data
    }

template <class T>
void Queue<T>::Add(T* y) {
    QueueNode<T>* temp = new QueueNode<T>(y);
    if (front == 0) front = rear = temp;//empty queue
    else rear = rear->link = temp; //attach node and update rear
}

template <class T>
T* Queue<T>::Delete()
//delete the first node in queue and return a pointer to its data
{
    if (front == 0) { QueueEmpty(); return 0; }
    QueueNode<T>* x = front;
    Employee *retvalue = front->data; //get data field of front node
    front = x->link;//remove front node
    delete x;
    return retvalue;//return pointer to data
}

template <class T>
void Queue<T>::QueueEmpty() {
    if (front == 0)
        cout << "Queue is empty" << endl;
    else
        cout << "Queue is non empty" << endl;
}

template <class T>
void Queue<T>::ShowQueue() {
    QueueNode<T>* p = front;
    while (p != NULL) {
        cout << *p->data;
        if (p->link != NULL)
            cout << " < ";
    }
}

```

```

        p = p->link;
    }
    cout << endl;
}
enum Enum {
    PushStack, PopStack, ShowStack, AddQueue, DeleteQueue, ShowQueue,
    Exit
};

void main() {
    Enum menu; // 메뉴
    int selectMenu, num;
    string eno, ename;
    bool result = false;
    Stack<Employee> st;
    Queue<Employee> qu;
    Employee* data;
    do {
        cout << "0.PushStack, 1.PopStack, 2.ShowStack, 3.AddQueue,
4.DeleteQueue, 5.ShowQueue, 6.Exit:: ";
        cin >> selectMenu;
        switch (static_cast<Enum>(selectMenu)) {
            case PushStack:
                cout << "사원번호 입력:: ";
                cin >> eno;
                cout << "사원 이름 입력:: ";
                cin >> ename;
                data = new Employee(eno, ename);
                st.Add(data);
                break;
            case PopStack:
                data = st.Delete();
                cout << *data;
                break;
            case ShowStack:
                st.Show();
                break;

```

```

        case AddQueue:
            cout << "사원번호 입력:: ";
            cin >> eno;
            cout << "사원 이름 입력:: ";
            cin >> ename;
            data = new Employee(eno, ename);
            qu.Add(data);
            break;
        case DeleteQueue:
            data = qu.Delete();
            cout << *data;
            break;
        case ShowQueue:
            qu.ShowQueue();
            break;

        case Exit: // 꼬리 노드 삭제
            break;
    }
} while (static_cast<Enum>(selectMenu) != Exit);
cin >> num;
}

```

4.10 Generalized list

Generalized list는 down link와 rlink를 사용하며 down link로 연결된 또 다른 generalized list를 만든다. link field가 2개가 있는 것은 doubly linked list와 유사하나 각 노드가 항상 down link를 갖지 않을 수 있기 때문에 doubly list와는 다르다. 오히려 5장에서 공부할 tree와 유사한 측면이 많다.

generalized list, A는 유한개의 sequence of elements인 $a_0, \dots, a_{n-1}, a_i$ 을 포함하는 데 각 element가 단순 item 또는 list일 수가 있다. generalized lists의 예제로서 $D = []$ 은 null or empty list이다. $A = [a, [b, c]]$ 는 길이가 2인 list이다. $B = [A, A, []]$ 는 길이가 3인 list이며 리스트 A를 포함한다. $C = [a, C]$ 는 길이가 2인 recursive list로서 두 번째 element인 C가 자기 자신을 가리키는 list이다. 1개 이상의 변수로 구성된 polynomial을 linked list로 표현하기 위한 자료구조로서 singly linked list를 사용하는 방법[1]을 살펴본다. struct를 사용하여 coef, expx, expy, expz를 노드에 포함하고 link를 포함한다. 만약 다항식에 x,y,z 이외의 w 변수를 추가로 사용한다면 expw를 추가해야 한다. polynomial에 사용되는 변수 개수에 따라 노드의 field가 추가되는 문제가 있다. 그런데 다항식은 어떤 항은 변수가 x만 있을 경우에 expy, expz, expw는 모두 0으로 표기해야 하는 불편이 있다.

$$P(x, y, z) = x^{10}y^3z^2 + 2x^8y^3z^2 + 3x^8y^2z^2 + x^4y^4z + 6x^3y^4z + 2yz$$

다항식에 포함되는 각 항의 변수 개수에 관계없이 각 노드는 4개의 field로 고정하여 구현하는 방법이 generalized list이다. 다항식을 표현하기 위해 먼저 변수 z를 사용하여 factoring out한 z의 다항식, 즉 $P(z) = Cz^2 + Dz$ 을 만든다. 다음에 변수 y에 대하여, 그 다음은 x에 대하여 factoring out을 한 표현식을 만든다.

$$P(x, y, z) = ((x^{10} + 2x^8)y + 3x^8y^2)z^2 + ((x^4 + 6x^3)y + 2y)z$$

$$P(x, y, z) = Cz^2 + Dz, \text{ where } C(x, y) = Ey^3 + Fy^2$$

각 polynomial을 표현하기 위한 각 노드는 link, exp를 포함하고 {variable, coefficient, variable}의 union field와 이를 구분하기 위한 tag로서 trio를 포함한다. polynomial node를 표현하기 위한 Polynode[1]는 다음과 같다.

```
enum Triple {var, ptr, no};
class PolyNode
{
    Polynode *next;
    int exp;
    Triple trio;
    union {
        char vble;
        PolyNode *down;
    };
};
```



```

        int coef;
    };
};

```

trio는 enum으로 3가지 tag에 따른 용도를 구분한다. trio == var이면 vble field로 사용하며 head node로 사용된다. trio == ptr이면 down field로 사용한다. trio == no이면 coef field로 사용한다. $P(x,y) = 3x^2y$ 는 그림 4.7과 같이 list를 만든다.

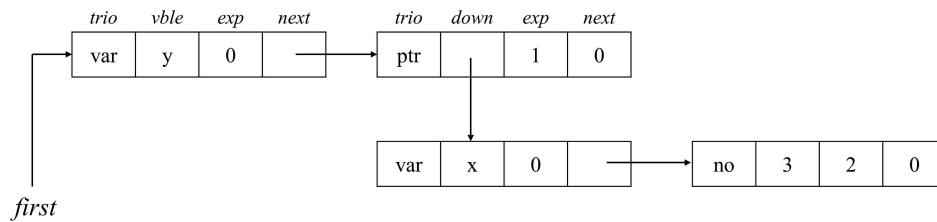


그림 4.7 Generalized List.

다항식 $P(x, y, z) = ((x^{10} + 2x^8)y + 3x^8y^2)z^2 + ((x^4 + 6x^3)y + 2y)z$ 에 대한 generalized list[1]는 그림 4.8과 같다. 그림 4.8의 list는 trio field를 생략하여 보여주고 있다.

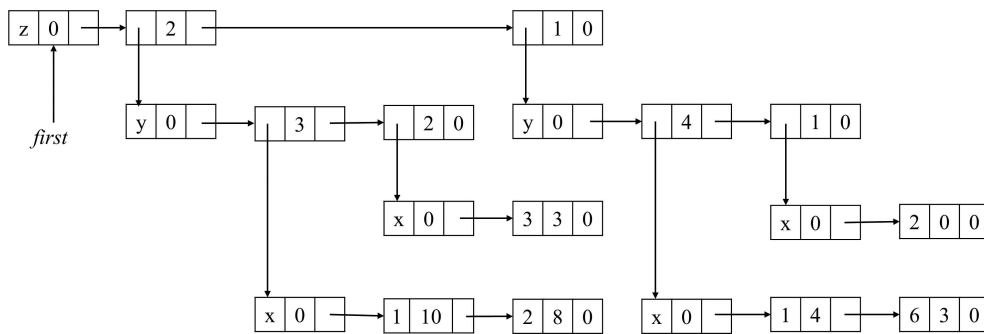


그림 4.8 다항식 표현 예.

좀 더 일반화하여 {tag, data/down, next}로 구성된 node struct를 사용한 generalized list[1]를 표현할 수 있다.

```

enum Boolean {FALSE, TRUE}
class GenList;

```

```

class GenListNode {
    friend class GenList;
private:
    GenListNode *next;
    Boolean tag;
    union {
        char data;
        GenListNode *down;
    };
};

class GenList {
public:
    // list manipulation operations
private:
    GenListNode *first;
};

```

generalized list는 list 그 자체가 recursive하게 정의되므로 copy하는 알고리즘도 recursive 함수로 작성하는 것이 자연스럽다. recursive algorithm은 2개의 함수로 구현하는 데 첫째 함수가 자기 자신을 호출하는 recursive function이며 이를 workhorse(driver에서 호출되는 함수로서 시키는 작업을 호출할 때마다 되풀이한다는 의미)라고 하며 private 함수로 구현한다. 두 번째 함수는 main()에서 호출되는 함수로서 driver라고 하며 public 함수로 구현한다. class GenList에서 private 함수로 GenListNode* Copy(GenListNode*);그리고 public 함수로 void Copy(const GenList&);가 있다[1].

```

//Driver
void GenList<T>::Copy(const GenList<T>& l) const
{ //Make a copy of l
    first = Copy(l.first);
}

//Workhorse
GenListNode<T>* GenList<T>::Copy(GenListNode<T> *p)
{//Copy the nonrecursive list with no shared sublists pointed at by p
    GenListNode<T> *q = 0;
    if(p) {

```

```

        q = new GenListNode<T>;
        q->tag = p->tag;
        if (p->tag) q->down = Copy(p->down);
        else q->data = p->data;
        q->next = Copy(p->next);
    }
    return q;
}

```

두 개의 generalized lists가 같은 지 검사하는 List Equality와 List Depth도 Driver와 Workhorse 함수를 사용한다.

```

//소스 코드4.8: Generalized List
//Devise a function that produces a gen list from the input l = (a,(b,c))
/*
소스 코드 실습 대상
1. copy하여 copy 전과 후의 결과를 출력
2. 2개의 gen list를 ==
3. depth 결과를 출력
4. reference count 사용하여 getnode, retnode 구현, gen list 반환 코드 구현 테스트
5. template version으로 만드는 것 - int data:를 T data:로 변경하고 T는 coef, exp로
   사용하는 구현한다
*/
#include <iostream>

using namespace std;
int GetData();

class GenList: // forward declaration

class GenListNode
{
    friend class GenList;
    friend int operator==(const GenList&, const GenList&);
    friend int equal(GenListNode*, GenListNode*);
}

```

```

private:
    GenListNode* link;
    bool tag; // 0 for data, 1 for dlink, 2 for ref
    union {
        int data;
        GenListNode* dlink;
        int ref;
    };
};

class GenList
{
    friend int operator==(const GenList&, const GenList&);
    friend int equal(GenListNode*, GenListNode*);
private:
    GenListNode* first;
    static GenListNode* av;
    GenListNode* Copy(GenListNode*);
    void Visit(GenListNode*); //driver
    int Depth(GenListNode*);
    GenListNode* MakeList(GenListNode*);
    void Delete(GenListNode*);
public:
    void Copy(const GenList&);
    void Visit(); //workhorse
    ~GenList();
    int Depth();
    void MakeList();
};

GenList::~~GenList()
//Each head node has a reference count. We assume first != 0
{
    Delete(first);
    first = 0;
}

void GenList::Copy(const GenList& l)

```

```

{
    first = Copy(l.first);
}

void GenList::Visit()//public function으로 구현
{
    cout << "<";
    Visit(this->first);
    cout << ">";
}
/*
GenList::~GenList( )//구현하는 실습을 한다.
{
    Delete(first);
    first = 0;
}
*/

int operator==(const GenList& l, const GenList& m)
{
    return equal(l.first, m.first);
}

//Workhorse - 반환시에 문제 발생함
void GenList::Delete(GenListNode* x)
{
    if (x->tag == 2) x->ref--;
    if (!x->ref)
    {
        GenListNode* y = x;
        while (y->link)
        {
            y = y->link;
            if (y->tag == 1) Delete(y->dlink);
        }
        y->link = av;
        av = x;
    }
}

```

```

    }
}

```

```

int GenList::Depth()
{
    return Depth(first);
}

```

```

void GenList::MakeList()
{
    first = MakeList(first);
}

```

//copy, visit, equal 등의 recursive 함수를 먼저 공부한 후에 MakeList()를 이해하는 것이 좋다

```

GenListNode* GenList::MakeList(GenListNode* s)
{

```

```

    GenListNode* p, * q, * r;
    bool tag_head = true;
    q = new GenListNode;
    r = q;
    while (int x = GetData())//if x == 0, no more entry
    {

```

if (x == 1) //make down link - recursive function 처리가 필요하다.

```

        {
            if (tag_head) { //x ==1이면 새로운 down link에서 리스트를

```

만든다.

```

                q->dlink = MakeList(q);
                q->tag = true;
                q->link = 0;
                tag_head = false;
                continue;
            }
            else
            {

```

```

        p = new GenListNode;
        q->link = p;
        p->tag = true;
        p->link = 0;
        q = p;
        q->dlink = MakeList(q);
    }
}
else { // x >= 2는 listnode의 데이터 값으로 리스트를 만든다
    if (tag_head) {
        q->data = x; q->tag = false; q->link = 0;
        tag_head = false;
    }
    else {
        p = new GenListNode;
        p->data = x; p->tag = false; p->link = 0;
        q->link = p;
        q = p;
    }
}
}
return r;
}

GenListNode* GenList::Copy(GenListNode* p) //private function
{
    GenListNode* q = 0;
    if (p) {
        q = new GenListNode;
        q->tag = p->tag;
        if (!p->tag) q->data = p->data;
        else q->dlink = Copy(p->dlink);
        q->link = Copy(p->link);
    }
    return q;
}

```

```

void GenList::Visit(GenListNode* p)
{
    if (p != nullptr) {
        cout << p->tag << " ";
        if (!p->tag)
            cout << p->data << " ";
        else {
            cout << " <";
            Visit(p->dlink);
            cout << "> ";
        }
        Visit(p->link);
    }
}

int equal(GenListNode* s, GenListNode* t)
{
    int x;
    if ((!s) && (!t)) return 1;
    if (s && t && (s->tag == t->tag)) {
        if (!s->tag)
            if (s->data == t->data) x = 1; else x = 0;
        else x = equal(s->dlink, t->dlink);
        if (x) return equal(s->link, t->link);
    }
    return 0;
}

int GenList::Depth(GenListNode* s)
{
    if (!s) return 0;
    GenListNode* p = s; int m = 0;
    while (p) {
        if (p->tag) {
            int n = Depth(p->dlink);
            if (m < n) m = n;
        }
    }
}

```



```

        p = p->link;
    }
    return m + 1;
}

int GetData() {
    cout << endl << "0: exit, 1: dlink, 2 정수입력: ";
    int n;
    cin >> n;
    return n;
}

GenListNode* GenList::av = 0;
int main(void)
{
    GenList l;
    GenList m;
    char select;
    int max = 0, x = 0;
    cout << "Select command m: make, c: copy, d: depth, v: visit, q : quit
=>";
    cin >> select;
    while (select != 'q')
    {
        switch (select)
        {
            case 'm':
                l.MakeList();
                break;
            case 'c':
                m.Copy(l);
                cout << "Copy Result" << endl;
                l.Visit();
                m.Visit();
                cout << endl;
                break;
            case 'd':

```

```

        cout << "depth: " << l.Depth();
        cout << endl;
    case 'v':
        cout << "tag : data (link)" << endl;
        l.Visit();
        cout << endl;
        break;
    case 'q':
        cout << "Quit" << endl;
        break;

    default:
        cout << "WRONG INPUT  " << endl;
        cout << "Re-Enter" << endl;
        break;
}
cout << "Select command m: make, c: copy, d: depth, v: visit, q
: quit =>";

    cin >> select;
}
system("pause");
return 0;
}

```

특정 sublist가 다른 list에 의해 referenced되고 있는 경우에 주어진 list의 삭제가 dangling reference를 초래할 수 있다. 각 list에 대한 head node를 두고 head node의 data 값이 피참조되고 있는 count field를 둔다. head node의 data/down field가 reference count를 기록하여 0이 아니면 해당 list를 delete 하지 않는다. GenListNode<T>에서 int ref:를 union에 포함하여 구현한다.

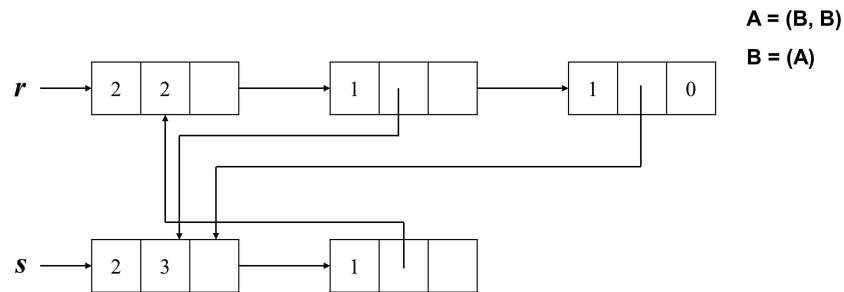


그림 4.9 GenList의 reference count.

list를 erase하는 recursive 알고리즘은 `GenList::~~GenList( )`의 body에서 `void GenList::Delete(GenListNode* x)`을 호출한다. generalized list에 대한 실습을 위해서는 list를 만드는 `void MakeList( )`에서 데이터를 입력받아 생성한다. generalized list의 생성도 recursive 알고리즘으로 구현한 소스 코드를 보고 실습을 해본다.

```
template <class T>
class GenListNode<T>
{
    friend class GenList<T>;
private:
    GenListNode<T> *next;
    int tag;//0 for data, 1 for down, 2 for ref
    union {
        T data;
        GenListNode<T> *down;
        int ref;
    };
};

//Gen List
//Devise a function that produces a gen list from the input l = (a,(b,c))
/*
소스 코드 실습 대상: generalized list를 구현하는 실습
1. copy하여 copy 전과 후의 결과를 출력
2. 2개의 gen list를 ==
3. depth 결과를 출력
4. reference count 사용하여 getnode, retnode 구현, gen list 반환 코드 구현 테스트
5. template version으로 만드는 것
*/
```

4.11 Heterogeneous lists

list의 노드가 다른 type을 가지면 heterogeneous list라고 한다. union을 사용하여 다른 타입의 노드들을 하나의 class definition으로 처리할 수 있다. union을 사용하여 노드 타입을 merge할 때 가장 큰 데이터 타입의 공간을 할당하는 것이 필요하다. $S(\)$ 를 해당 타입의 공간 크기이다.

$$\begin{aligned} S(\text{CombinedNode}) &= S(\text{Data}) + S(\text{CombinedNode} *) \\ &= S(\text{int}) + \max(S(\text{char}), S(\text{int}), S(\text{float})) + \\ &\quad S(\text{CombinedNode} *) \end{aligned}$$

```
struct Data
{
    int id;//id = 0,1,2 for char, int, float
    union {
        int i;
        char c;
        float f;
    };
};
```

```
class CombinedNode
{
    friend class List;
    friend class ListIterator;
private:
    Data data;
    CombinedNode *link;
};
```

```
class List
{
    friend class ListIterator;
```

```

public:
//List manipulation operations follow
private:
    CombinedNode *first;
};

class Node {
friend class List;
friend class ListIterator;
protected:
    Node *link;
    virtual Data GetData() = 0;
};

template<class Type>
class DerivedNode: public Node {
friend class List;
friend class ListIterator;
public:
    DerivedNode(Type item): data(item) {link = 0;};
private:
    Type data;
    Data GetData();
};

Data DerivedNode<char>::GetData()
{
    Data t;
    t.id = 0;
    t.c = data;
    return t;
}

```

- (1) Abstract class Node를 pure virtual 함수로 정의한다. Node *link;는 다른 데이터 타입에 대하여 공통이므로 class Node에 포함한다.

(2) DerivedNode<Type>을 Node의 subclass로 정의한다. Type data:를 포함한다.

(3) DerivedNode<Type>::GetData()를 구현한다. GetData() 함수 구현은 char, int, float 등에 대하여 각각 구현되어야 한다.

(4) List, ListIterator class는 union을 사용하는 struct Data에 영향을 받지 않는다.

상속에 의한 dynamic typing으로 Node*에 대한 pointer는 DerivedNode<char>, DerivedNode<int>, DerivedNode<float>의 노드로 binding된다[1].

```
//소스 코드4.9: Heterogeneous Linked List
#include <iostream>
#include <string>
#include <cctype>

using namespace std;
enum boolean { FALSE, TRUE };
struct Data
{
    int id; //id = 0, 1, 2 if the node contains char, int, float
    union {
        int i;
        char c;
        float f;
    };
    bool operator==(Data const d);
};

/*
class CombinedNode {
//use union to merge different node types into one class definition
    friend class List;
    friend class ListIterator;
private:
    Data data;
    CombinedNode* link;
};
class List {
    friend class ListIterator;
public:
private:
    CombinedNode* first;
```

```

};
*/

class Node {
    friend class List;
    friend class ListIterator;
protected:
    Node* link;
    virtual Data GetData( ) = 0;
};

template<class Type>
class DerivedNode : public Node {
    friend class List;
    friend class ListIterator;
public:
    DerivedNode(Type item) : data(item) { link = 0; }
private:
    Type data;
    Data GetData( );
};

class List {
    friend class ListIterator;
public:
    List( ) { first = 0; };
    ~List( ) { };
    void Add( );
    void Delete( );
    Node* Search(Data);
private:
    Node* first;
};

class ListIterator {
public:
    ListIterator(const List& l) : list(l), current(l.first) { };
    ListIterator& operator ++( );
    ListIterator operator ++(int);
    Data* First( );
    Node* Next( );
    bool NotNull( );
    bool NextNotNull( );
};

```

```

        Data GetCurrent( );
        Node* CurrentPointer( );
private:
        const List& list;
        Node* current;
        Data temp;
};

bool Data::operator==(Data const d)
{
    switch (d.id)
    {
        case 0:
            if (this->c == d.c)
                return TRUE;
            break;
        case 1:
            if (this->i == d.i)
                return TRUE;
            break;
        case 2:
            if (this->f == d.f)
                return TRUE;
            break;
    }
    return FALSE;
}

Data DerivedNode<char>::GetData( ) {
    Data t; t.id = 0; t.c = data; return t;
}

Data DerivedNode<int>::GetData( ) {
    Data t; t.id = 1; t.i = data; return t;
}

Data DerivedNode<float>::GetData( ) {
    Data t; t.id = 2; t.f = data; return t;
}

void List::Add( )
{
    int value;

```



```

float num;
char c;
cout << "Please input data type. 0 = char, 1 = int, 2 = float" << endl;
cin >> value;
if (!first)
    switch (value)
    {
    case 0:
        cin >> c;
        first = new DerivedNode<char>(c);
        break;
    case 1:
        cin >> value;
        first = new DerivedNode<int>(value);
        break;
    case 2:
        cin >> num;
        first = new DerivedNode<float>(num);
        break;
    default:
        break;
    }
else
{
    switch (value)
    {
    case 0:
    {
        cin >> c;
        Node* n = new DerivedNode<char>(c);
        n->link = first;
        first = n;
        break;
    }
    case 1:
    {
        cin >> value;
        Node* na = new DerivedNode<int>(value);
        na->link = first;
        first = na;
        break;
    }
    case 2:

```

```

        {
            cin >> num;
            Node* nb = new DerivedNode<float>(num);
            nb->link = first;
            first = nb;
            break;
        }
        default:
            break;
    }
}

}

void List::Delete( )
{
    int value;
    char c;
    float num;
    Data tmp;

    if (first == NULL) cout << " List is empty" << endl;
    else
    {
        cout << "Please input data type. 0 = char, 1 = int, 2 = float" << endl;
        cin >> value;
        cin.clear( );
        switch (value)
        {
            case 0:
                cin >> c;
                tmp.c = c;
                tmp.id = 0;
                break;
            case 1:
                cin >> value;
                tmp.i = value;
                tmp.id = 1;
                break;
            case 2:
                cin >> num;
                tmp.f = num;
                tmp.id = 2;

```

```

        break;
    default:
        break;
    }
    Node* s = Search(tmp);
    if (s == first && !(s->link->GetData( ) == tmp))
    {
        Node* tmp;
        tmp = s;
        s = s->link;
        first = s;
        delete tmp;
    }
    else if (s)
    {
        Node* tmp;
        tmp = s->link;
        s->link = s->link->link;
        delete tmp;
    }
}

Node* List::Search(Data tmp)
{
    ListIterator li(*this);
    while (li.NotNull( ))
    {
        if (li.GetCurrent( ) == tmp)
            return li.CurrentPointer( );
        else if (li.Next( )->GetData( ) == tmp)
            return li.CurrentPointer( );
        ++li;
    }
    return NULL;
}

Data* ListIterator::First( ) {
    if (list.first) {
        temp = list.first->GetData( );
        return &temp;
    }
    return 0;
}

```

```

}

Data ListIterator::GetCurrent( ) {
    return current->GetData( );
}

Node* ListIterator::Next( )
{
    return current->link;
}

Node* ListIterator::CurrentPointer( )
{
    return current;
}

ListIterator& ListIterator:: operator ++( )
{
    current = current->link;
    return *this;
}

ListIterator ListIterator:: operator ++(int)
{
    ListIterator old = *this;
    current = current->link;
    return old;
}

bool ListIterator::NotNull( )
{
    if (current) return TRUE;
    else return FALSE;
}

bool ListIterator::NextNotNull( )
{
    if (current->link) return TRUE;
    else return FALSE;
}

ostream& operator<<(ostream& os, Data& retvalue)
{
    switch (retvalue.id)

```

```

    {
    case 0:
        cout << retvalue.c;
        break;
    case 1:
        cout << retvalue.i;
        break;
    case 2:
        cout << retvalue.f;
        break;
    default:
        break;
    }
    return os;
}

void PrintAll(const List& l)
{
    ListIterator li(l);
    if (!li.NotNull( )) return;
    Data retvalue = *li.First( );
    cout << retvalue;
    if (li.NextNotNull( ))
        cout << " + ";
    while (li.NextNotNull( ) == true) {
        ++li; //current를 증가시킴
        //retvalue = retvalue + *li.Next( );
        retvalue = li.GetCurrent( ); //현재 current가 가르키는 node의 값을 가져옴
        cout << retvalue;
        if (li.NextNotNull( ))
            cout << " + ";
    }
}

//float sum(l); //sum of all floating points
int main(void)
{
    List l;
    //implement heterogeneous linked stacks and queues
    char select;
    int max = 0, x = 0;
    cout << "Select command a: add data, d: delete data, p: print all, q : quit =>";
    cin >> select;

```

```

while (select != 'q')
{
    switch (select)
    {
        case 'a':
            l.Add( );
            break;
        //case 'c': l.Copy( ); //구현실습
        case 'd':
            l.Delete( );
            break;
        //case 'e': l.Equal( ); //구현실습
        case 'p':
            cout << "print all: ";
            PrintAll(l);
            cout << endl;
            break;
        case 'q':
            cout << "Quit" << endl;
            break;

        default:
            cout << "WRONG INPUT  " << endl;
            cout << "Re-Enter" << endl;
            break;
    }
    cout << "Select command a: add data, d: delete data, p: print all, q : quit
=>";

    cin >> select;
}
system("pause");
return 0;
}

```

연습 문제

1. 다음 function을 구현할 때 다음 조건을 모두 만족해야 한다.

1.1) template 사용

1.2) class Term을 사용해야 한다.

```
template<class T> class Term {
public:
    //All members of Term are public by default
    T coef; //coefficient
    T exp; //exponent
    Term( ) { coef = 0; exp = 0; }
    Term(T c, T e) :coef(c), exp(e) { }
    Term Set(int c, int e) { coef = c; exp = e; return *this; };
};
```

1.3) iterator를 사용하지 않는다. circular list로서 head node를 사용한다.
circular list의 last 변수가 마지막 노드를 가리킨다.

```
template <class Type>
class CircList {
    friend class CircListIterator<Type>;
public:
    CircList( ) { first = new ListNode<Type>; first->link = first; last = first; };
    CircList(const CircList&);
    ~CircList( );
    void Attach(Type);
private:
    ListNode<Type>* first;
    ListNode<Type>* last;
    static ListNode<Type>* av;
};
```

1.4) available list를 사용하고 getnode(), retnode()를 사용한다.

1.5) cin >> a:로 다항식을 입력받는다. 지수는 내림차순으로 입력되어야 하고 입력된 지수가 이미 입력된 지수보다 크면 다시 입력받는다. 지수가 -1이면 입력받

는 것을 종료한다. 입력되는 항은 10개 이상으로 하되 지수는 내림차순으로 하고 계수는 임의 값 입력하는데 난수 생성하여 자동 입력한다(화면에서 입력하지 않게 구현한다).

1.6) `cout<<d;`는 $a(x) = 5x^3 + 4x^2 + 5$ 로 출력한다. `operator<<(Polynomial &p)`는 `operator<<(CircList<> &c)`를 호출하여 구현한다.

1.7) $a*b$ 는 $x+y$ 를 호출하여 답을 구한다.

```
viod func( )
{
    // d = a+ b* c;
    polynomial a, b, c, d;
    cin >> a; // read and create polynomials
    cin >> b;
    cin >> c;
    {
        polynomial t;
        t = b * c
        cout <<"a(x) = " << a << ", b(x) = " <<b << ", a(x) * b(x) = " <<t;
        d = a + t;
    } //t의 소멸차 호출한다 => retNode( ) 호출한다
    cout <<"a(x) + b(x) * c(x) = " << d;
}
```